

线性表

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 线性表的基本概念
- 顺序表
 - 初始化、创建、插入、删除
 - 遍历查找、遍历显示、获取数据
 - 合并
- 单链表
 - 初始化；创建：头插法、尾插法；插入、删除
 - 遍历查找、遍历显示、获取数据销毁、求差集

内容要点

- 循环链表
 - 约瑟夫环
- 双向链表
 - 插入、删除
- 链表的应用
 - 逆置
 - 倒数第 k 个元素

目录

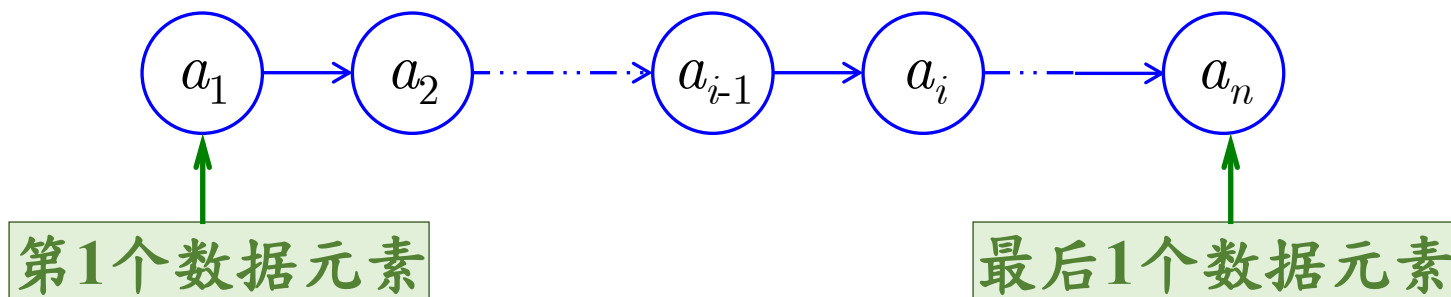
1	基本概念
2	顺序表
3	单链表
4	循环链表
5	双向链表

线性结构

- 特征

- 有而且只有一个“第一元素”；
- 有而且只有一个“最后元素”；
- 除第一元素之外，其他元素都有唯一的直接前驱；
- 除最后元素之外，其他元素都有唯一的直接后继。

- 线性表是一种常用的简单的线性结构。



线性表

- 线性表 (Linear List)
- 定义：具有相同类型数据元素的有限序列

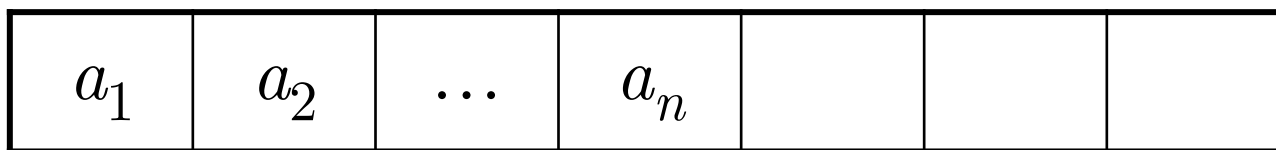
$$\mathbf{D} = \{ a_i \mid a_i \in \text{ElemSet}, \\ i=1, 2, \dots, n, n \geq 0 \}$$

$$(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$$

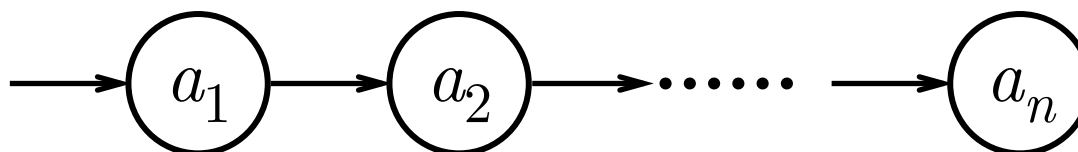
- 其中， i 是数据元素 a_i 在线性表 $\mathbf{D}$ 中的位序；
- n 是线性表的长度，即元素的个数；当 $n=0$ 时，称为空表。

线性表

- 顺序存储结构——由位置描述逻辑关系



- 链式存储结构——由指针描述逻辑关系



目录

1	基本概念
2	顺序表
3	单链表
4	循环链表
5	双向链表

顺序表：存储结构

- 用一组地址连续的存储单元存储线性表的数据元素

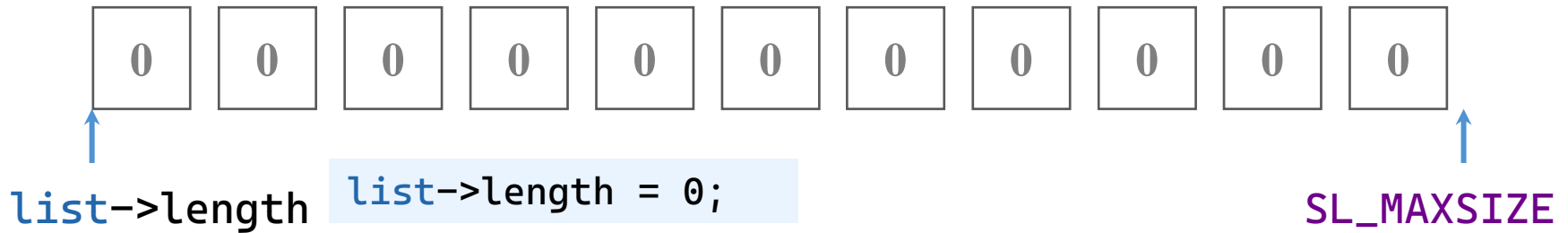
下标	0	1	...	$n-2$	$n-1$		
存储内容	a_1	a_2	...	a_{n-1}	a_n		
存储地址	k_0	k_1	...	k_{n-2}	k_{n-1}		

- 顺序表存储结构

```
#define SL_MAXSIZE 100
typedef struct SeqList {
    int data[SL_MAXSIZE];
    int length;
} SeqList;
```

顺序表：初始化

- 初始化：长度清零



```
void initList(SeqList* list) {  
    if (list != NULL) {  
        list->length = 0;  
    }  
}
```

顺序表：创建

currentValue

```
scanf("%d", &currentValue)
```

主体步骤

```
list->data[list->length] = currentValue;
```

```
list->length++;
```

```
list->length: 0    initList(list);
```

SL_MAXSIZE

currentValue

```
list->data[list->length] = currentValue;
```

```
list->length++;
```

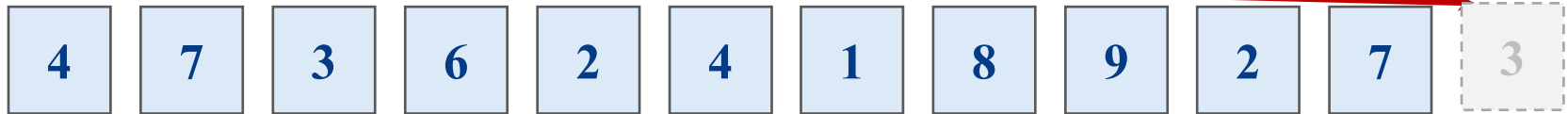
```
list->length: 1
```

SL_MAXSIZE

顺序表：创建

currentValue

3



边界判定

SL_MAXSIZE : 11

```
if (list->length >= MAX_SIZE)
    break;
```

list->length : 11

```
void createList(SeqList* list) {
    int currentValue = 0;
    initList(list);
    while (scanf("%d", &currentValue) == 1) {
        list->data[list->length] = currentValue;
        list->length++;
    }
}
```

顺序表：插入

• 顺序表的插入

– 将插入位置后的元素后移，再把新值赋值给插入位置

▪ 必须从尾部逆序，否则旧值会被覆盖，永久丢失

– 顺序表长度自增1。

```
for (i = list->length  
- 1; i >= index; i--)
```

`list->length: 6`

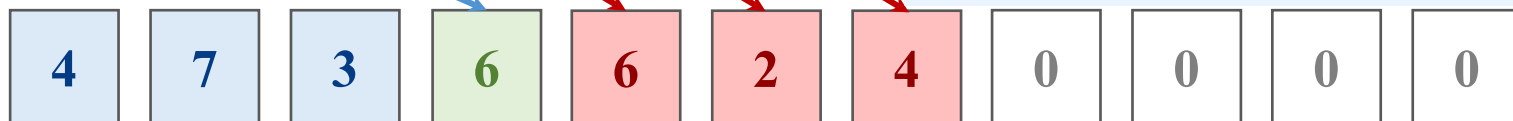
`list->length++;`

主体步骤



`index: 3`

`list->data[i+1] = list->data[i];`



data

9

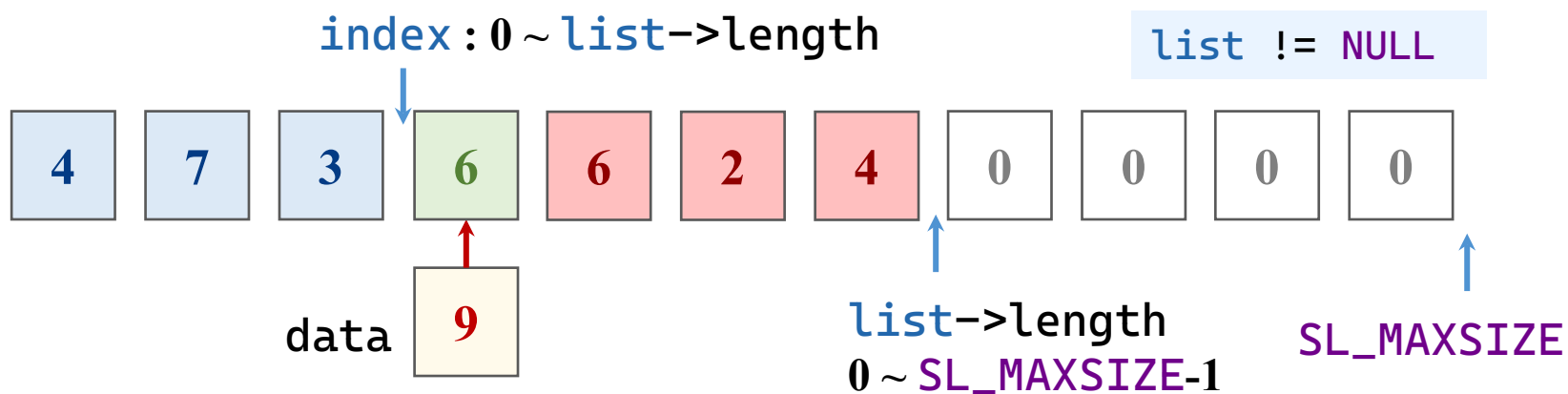
`list->data[index] = data;`

SL_MAXSIZE

顺序表：插入

• 顺序表的插入

边界判定



```

int insertList(const SeqList* list, const int index,
const int data) {
    int i = 0;
    if (list == NULL)
        return NULL_LIST;
    if (list->length >= MAX_SIZE)
        return ERROR;
    if (index < 0 || index > list->length)
        return NOT_FOUND;
    for (i = list->length - 1; i >= index; i--) {
        list->data[i + 1] = list->data[i];
    }
    list->data[index] = data;
    list->length++;
    return OK;
}

```

顺序表：删除

• 顺序表的删除

- 将删除位置的值赋给变量，删除位置后的元素前移
 - 必须从插入处顺序，否则旧值会被覆盖，永久丢失
- 顺序表长度自减1。

data **9**

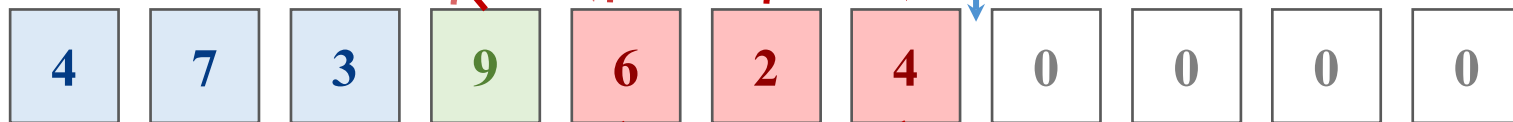
```
*data = list->data[index];
```

主体步骤

```
for (i = index; i < list->length - 1; i++)
```

list->length : 6

```
list->length--;
```



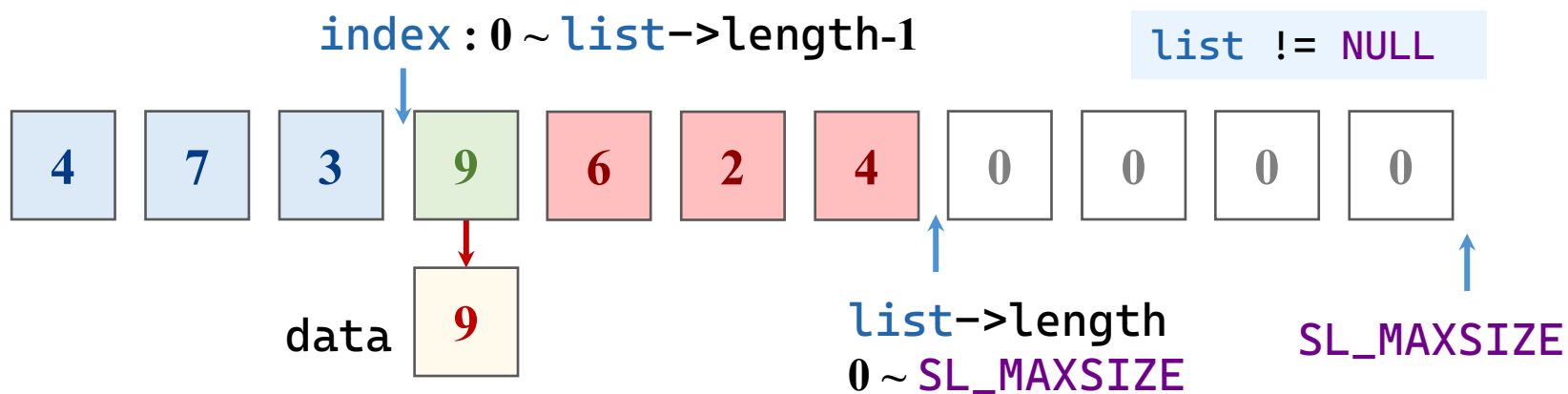
```
list->data[i] = list->data[i+1];
```



顺序表：删除

• 顺序表的删除

边界判定



```

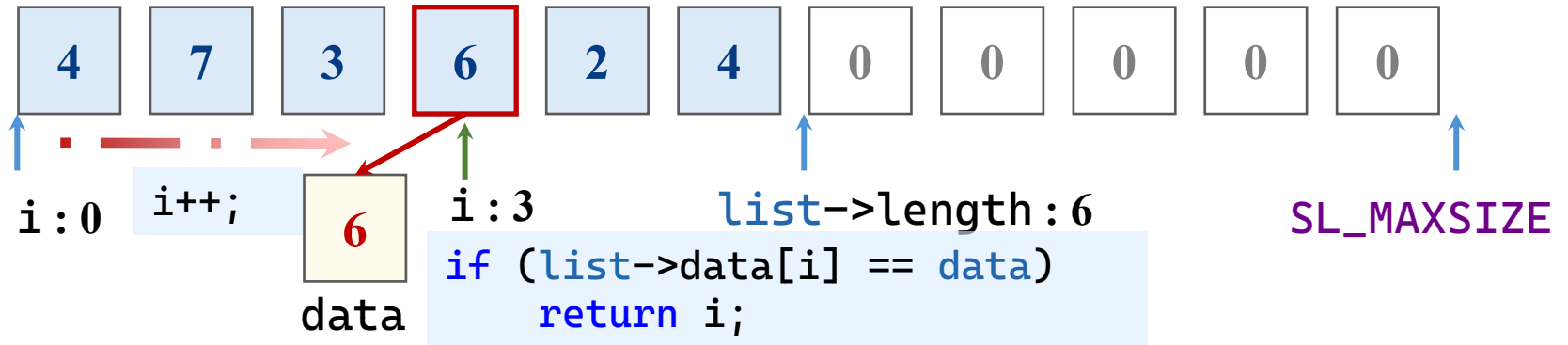
int deleteList(SeqList* list, int index, int* data) {
    int i = 0;
    if (list == NULL || data == NULL)
        return NULL_LIST;
    if (list->length <= 0)
        return ERROR;
    if (index < 0 || index >= list->length)
        return NOT_FOUND;

    *data = list->data[index];
    for (i = index; i < list->length - 1; i++)
        list->data[i] = list->data[i + 1];
    list->length--;
    return OK;
}

```

顺序表：遍历查找

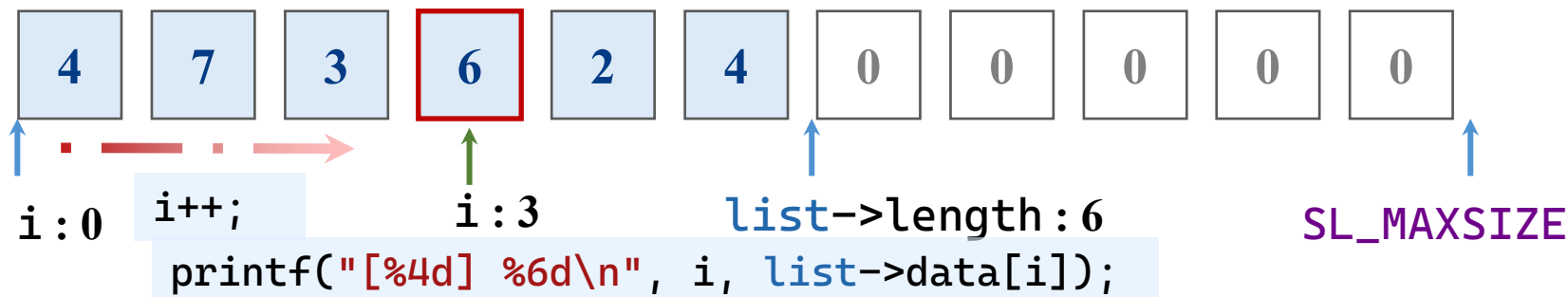
• 顺序表：遍历查找



```
int locate(const SeqList* list, int data) {  
    int i = 0;  
    if (list == NULL)  
        return NOT_FOUND;  
    for (i = 0; i < list->length; i++)  
        if (list->data[i] == data)  
            return i;  
    return NOT_FOUND;  
}
```

顺序表：遍历显示

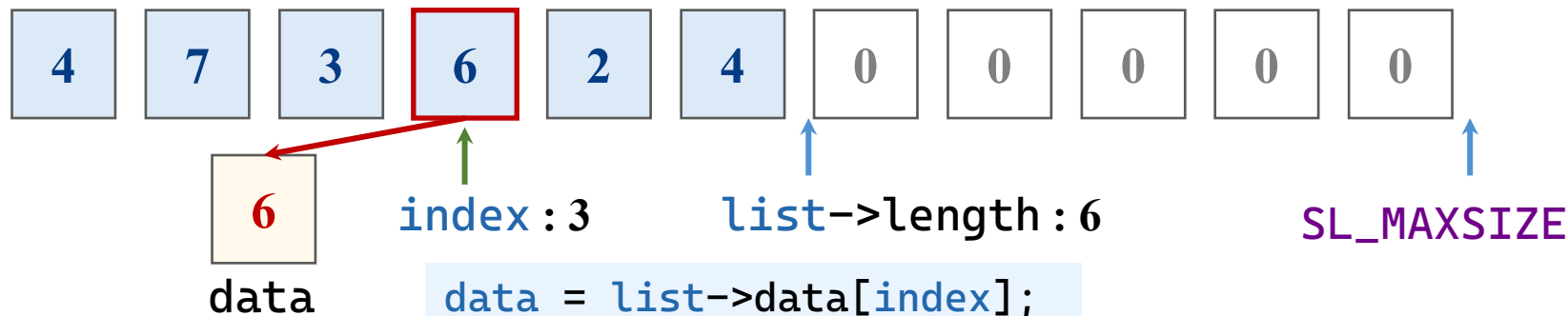
• 顺序表：遍历显示



```
int displayList(const SeqList* list) {
    int i = 0;
    if (list == NULL || list->length <= 0)
        printf("这是个空表! \n");
    else
        for (i = 0; i < list->length; i++)
            printf("[%4d] %6d\n", i, list->data[i]);
    return OK;
}
```

顺序表：获取数据

- 顺序表：根据索引获取数据



```
int getData(const SeqList* list, int index, int* data) {  
    if (list == NULL || data == NULL)  
        return NULL_LIST;  
    if (index < 0 || index >= list->length)  
        return NOT_FOUND;  
    *data = list->data[index];  
    return OK;  
}
```

顺序表：合并数据

• 合并两个升序表

```
if (listA->data[i] <= listB->data[j]) {  
}
```



顺序表：合并数据

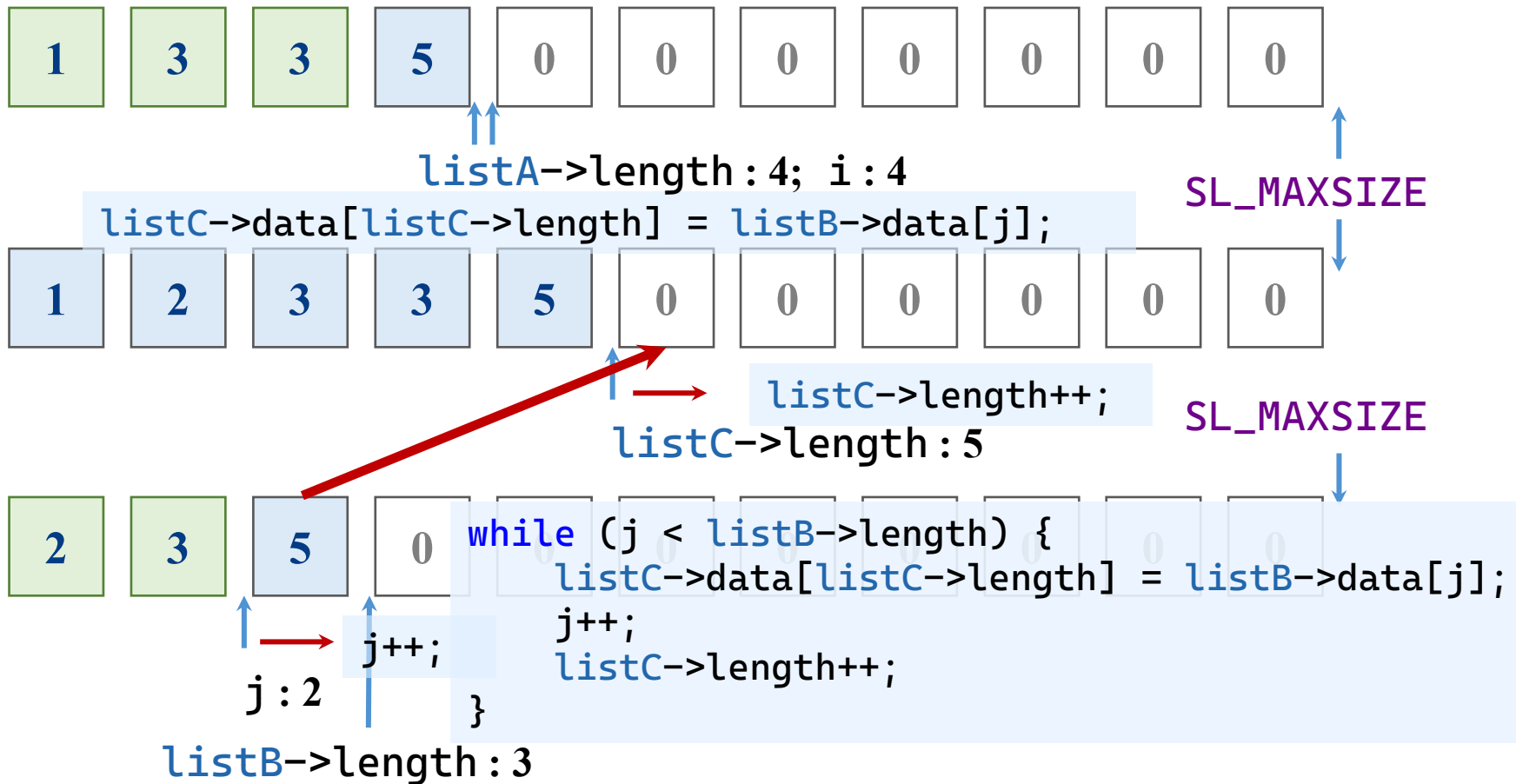
- 谁小就取谁

```
if (listA->data[i] <= listB->data[j])  
else {  
}
```



顺序表：合并数据

- 一方取完，没取完的一方继续取完



顺序表：合并数据

- 合并两个升序表
- 基本操作
 - 从头到尾遍历两个基础表，谁小就取谁
 - 取中的一方往前进，合并表也往前进
 - 没有取完的一方复制到合并表末尾
- 前后操作
 - 判断两个合并表长度之和是否越界
 - 升序排列两个基础表
 - 初始化合并表

```

int bubbleSort(SeqList* list) {
    int i = 0;
    int size = 0;
    int temp = 0;
    if (list == NULL)
        return NULL_LIST;
    for (size = list->length; size > 1; size--) {
        for (i = 1; i < size; i++) {
            if (list->data[i - 1] > list->data[i]) {
                temp = list->data[i - 1];
                list->data[i - 1] = list->data[i];
                list->data[i] = temp;
            }
        }
    }
    return OK;
}

```

```

int mergeList(SeqList* listA, SeqList* listB, SeqList* listC) {
    int i = 0;
    int j = 0;
    if (listA == NULL || listB == NULL || listC == NULL)
        return NULL_LIST;
    if (listA->length + listB->length > MAX_SIZE)
        return ERROR;
    bubbleSort(listA);
    bubbleSort(listB);
    initList(listC);
    while (i < listA->length && j < listB->length) {
        if (listA->data[i] <= listB->data[j]) {
            listC->data[listC->length] = listA->data[i];
            i++;
        } else {
            listC->data[listC->length] = listB->data[j];
            j++;
        }
        listC->length++;
    }
}

```

```
while (i < listA->length) {  
    listC->data[listC->length] = listA->data[i];  
    i++;  
    listC->length++;  
}  
  
while (j < listB->length) {  
    listC->data[listC->length] = listB->data[j];  
    j++;  
    listC->length++;  
}  
  
return OK;  
}
```

时间复杂度分析

- 冒泡排序法的时间复杂度 $O(n^2)$ 。
 - 两个线性表的时间复杂度 $O(m^2)$, $O(n^2)$
 - 用快速排序或归并排序，排序复杂度为 $O(m\log_2 m + n\log_2 n)$
- 顺序表合并
 - 这3个循环整体 i 和 j 是单调递增的，最多遍历 $m+n$ 次。
 - 时间复杂度 $O(m+n)$
- 总体时间复杂度
 - $T(m, n) = O(m^2) + O(n^2) + O(m+n) = O(m^2) + O(n^2)$

为数据结构书写测试程序

- 学完一个数据结构时，应书写测试程序，整理知识
- 书写一个菜单项程序
- 完善数据结构的各种操作
 - 先画图理解操作的过程
 - 再书写程序将图翻译为代码
 - 调试代码
- 优化输入输出，便于使用

```

#include <stdio.h>
#define MAX_SIZE 100
#define SL_MAXSIZE 100
#define OK 0
#define ERROR -1
#define NOT_FOUND -2
#define NULL_LIST -3
void clearInputLine(void) {
    int ch = 0;
    while ((ch = getchar()) != '\n' && ch != EOF) {
    }
}
int showMenu(void) {
    int choice = 0;
    printf("*****\n");
    printf("*          2.2 线性表的顺序存储          *\n");
    printf("*****\n");
    printf("    1.新建数据表\n");
    printf("    2.查找\n");
    printf("        21.按序号查找\n");
    printf("        22.按内容查找 [算法2.1]\n");
    printf("    3.插入 [算法2.2]\n");
    printf("    4.删除 [算法2.3]\n");
    printf("    5.合并 [算法2.4]\n");
    printf("    6.显示内容\n");
    printf("    7.退出\n");
}

```

```

printf("=====\\n");
printf("请选择: ");
scanf("%d", &choice);
return choice;
}
int main(void) {
    int ch = 0;
    int i = 0;
    int e = 0;
    SeqList listA, listB, listC;
    initList(&listA);
    initList(&listB);
    initList(&listC);
    while (ch != 7) {
        ch = showMenu();
        switch (ch) {
            case 1:
                createList(&listA);
                break;
            case 21:
                printf("请输入要查找的序号: ");
                scanf("%d", &i);
                if (getData(&listA, i, &e) == OK)
                    printf("第%d个元素为%d。\\n", i, e);
                else
                    printf("第%d个元素不存在。\\n", i);
        }
    }
}

```

```

        break;
    case 22:
        printf("输入要查找的元素值: ");
        scanf("%d", &e);
        i = locate(&listA, e);
        if (i != NOT_FOUND)
            printf("值为%d的元素第1次出现位置为%d。\\n", e, i);
        else
            printf("值为%d的元素并未在顺序表中出现\\n", e);
        break;
    case 3:
        printf("输入要插入的元素的值: ");
        scanf("%d", &e);
        printf("输入要插入的元素的位置: ");
        scanf("%d", &i);
        if (insertList(&listA, i, e) == OK)
            printf("插入成功! \\n");
        else
            printf("插入失败! 表已满或者插入位置错误。\\n");
        break;
    case 4:
        printf("输入要删除的元素的位置: ");
        scanf("%d", &i);
        if (deleteList(&listA, i, &e) == OK)
            printf("删除成功! 删去的值为%d。\\n", e);
        else

```

```

        printf("删除失败！表为空或者删除位置错误。\\n");
    break;
case 5:
    printf("主表为原表，现在输入副表：\\n");
    createList(&listB);
    initList(&listC);
    if (mergeList(&listA, &listB, &listC) == OK) {
        printf("主表：\\n");
        displayList(&listA);
        printf("副表：\\n");
        displayList(&listB);
        printf("合并后的表：\\n");
        displayList(&listC);
    } else {
        printf("合并失败！表长超出上限或者输入有误。\\n");
    }
    break;
case 6:
    printf("主表：\\n");
    displayList(&listA);
    break;
case 7:
    printf("结束程序。\\n");
    break;
default:
    printf("输入错误！请重新输入！\\n\\n");

```

```
        break;
    }

    if (ch != 7) {
        printf("单击空格键继续...\n");
        clearInputLine();
    }
}

clearInputLine();
return 0;
}
```

顺序表的主要特点

- 存储空间利用率高：只存储元素值。
- 随机存取
 - 可以通过计算来确定顺序表中存储地址
 - 第 i 个数据元素的地址是： $L_i = L_0 + (i - 1) \cdot m$
 - 其中， L_0 为第一个数据元素的存储地址， m 为每个数据元素所占用的存储单元数。
- 插入和删除数据元素会引起大量结点移动。

目录

	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表
	6	链表的应用

链表

- 链表是一种能够动态进行存储分配的数据结构。当需要插入数据元素时，申请一个结点的地址单元，并将该单元链入线性表中。
- 定义
 - 链表：以结点序列表示的线性表。
 - 结点：一个数据元素在内存中的映象（由连续的若干字节组成，该区域正好可以存储一个数据元素），包括数据域和指针域。
 - 数据域：存储数据元素的内容。
 - 指针域：存储后继元素的首地址。

链表

• 定义

- 线性链表（单链表）：链表的每个结点只包含一个指针域。
- 头结点：附加在第一个数据元素之前的结点，该结点的数据域一般为“空”、指针域存放第一个数据元素的地址。
- 头指针：线性链表中第一个结点或头结点的存储地址，它是访问链表的起始点。
- 动态链表：采用动态地址分配方法建立的线性链表（指针型描述）。

链表

- 线性链表操作要点：

- 访问链表，只能从头指针开始，沿着指针*next的指向进行访问(不能逆向)。
- 头指针不能丢——丢了，链表就没了。
- 在断开链表之前，注意保存后段链表的“头指针”，否则后段链表将丢失。

链表：数据结构

- 结点的存储结构

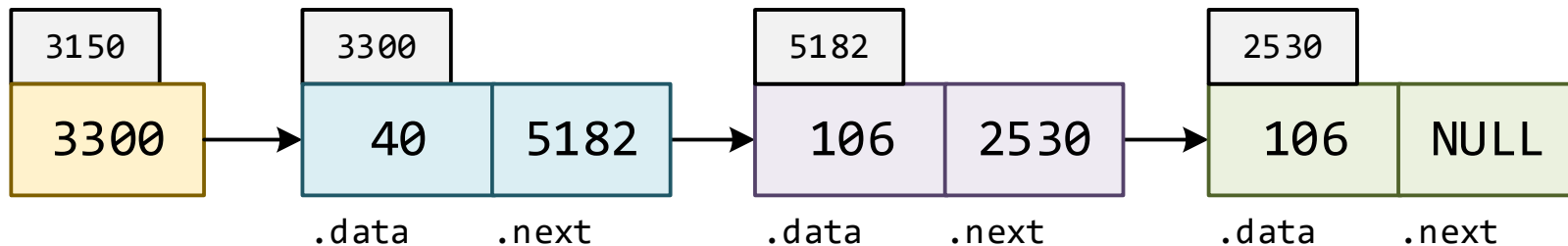
- 数据（data是数据域的示例）
- 下一结点指针 `p->next = NULL;`

```
typedef struct Node {  
    int data;  
    struct Node* next;  
} Node;  
typedef Node* LinkList;
```

- 链表是节点的指针

- 链表头：指向第一个结点的指针
- 链表中：next成员指向下一个结点
- 链表尾：next成员置为空

```
LinkList listA;
```



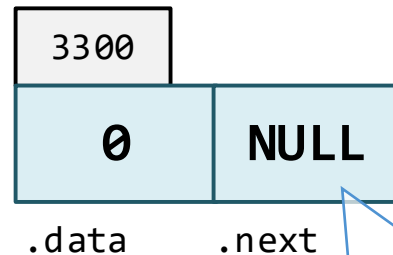
链表：创建结点并链接到另一结点

- 新建结点

```
node *p = (node*)malloc(sizeof(node));
```

- 赋初始值

- next初始值一般赋值为NULL
- 成员初始值一般按惯例赋值



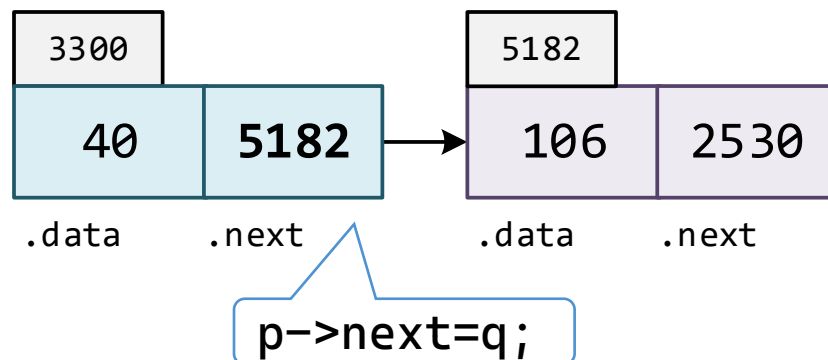
```
p->next=NULL;
```

```
p->data = 0;  
p->next = NULL;
```

- 形成前后顺序

- next赋值为另一结点的地址

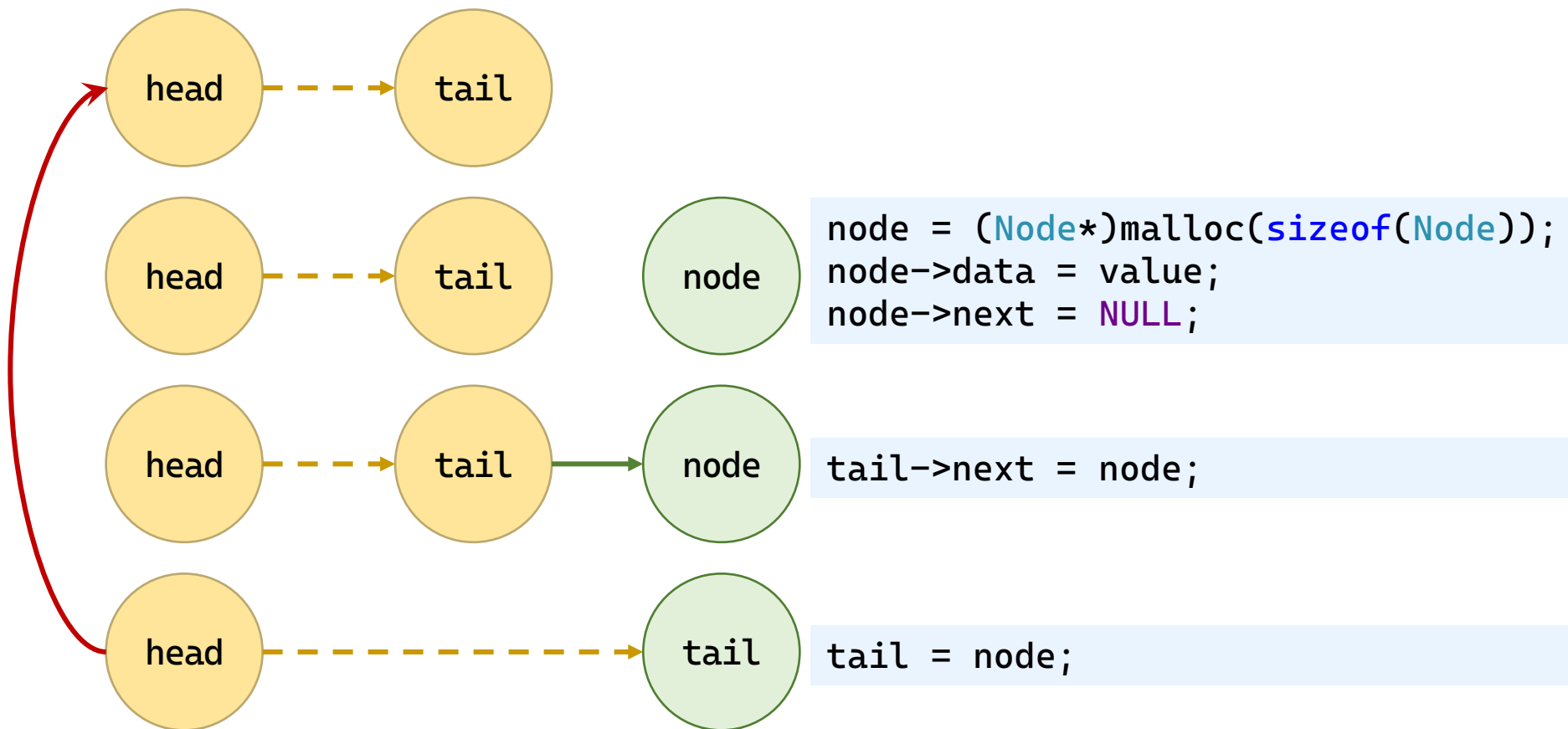
```
p->data = 40;  
p->next = q;
```



创建列表：尾插法

• 图示

主体步骤



链表：创建列表（尾插法）

• 初步代码

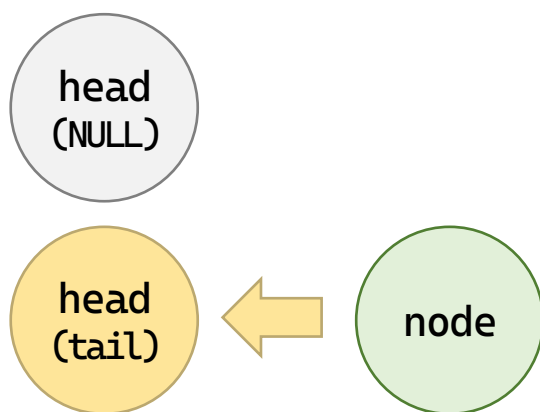
```
LinkedList create_from_tail(void) {  
    LinkedList head = NULL;  
    Node* tail = NULL, * node;  
    int value;  
    printf("请输入元素，输入非数字结束：\n");  
    while (scanf("%d", &value) == 1) {  
        node = (Node*)malloc(sizeof(Node));  
        node->data = value;  
        node->next = NULL;  
        tail->next = node;  
        tail = node;  
    }  
    return head;  
}
```

链表：创建列表（尾插法）

• 图示：链表还没建立的时候

特殊处理

1. head 为 NULL 时



2. node 生成失败时，报错、销毁

```
if (node == NULL) {  
    printf("内存分配失败。\\n");  
    free_list(&head);  
    clear_input_line();  
    return NULL;  
}
```

```
if (head == NULL) {  
    head = tail = node;  
} else {  
    tail->next = node;  
    tail = node;  
}
```

```

LinkedList create_from_tail(void) {
    LinkedList head = NULL;
    Node* tail = NULL, * node;
    int value;
    printf("请输入元素，输入非数字结束：\n");
    while (scanf("%d", &value) == 1) {
        node = (Node*)malloc(sizeof(Node));
        if (node == NULL) {
            printf("内存分配失败。\\n");
            free_list(&head);
            clear_input_line();
            return NULL;
        }
        node->data = value;
        node->next = NULL;
        if (head == NULL) {
            head = tail = node;
        } else {
            tail->next = node;
            tail = node;
        }
    }
    clear_input_line();
    return head;
}

```

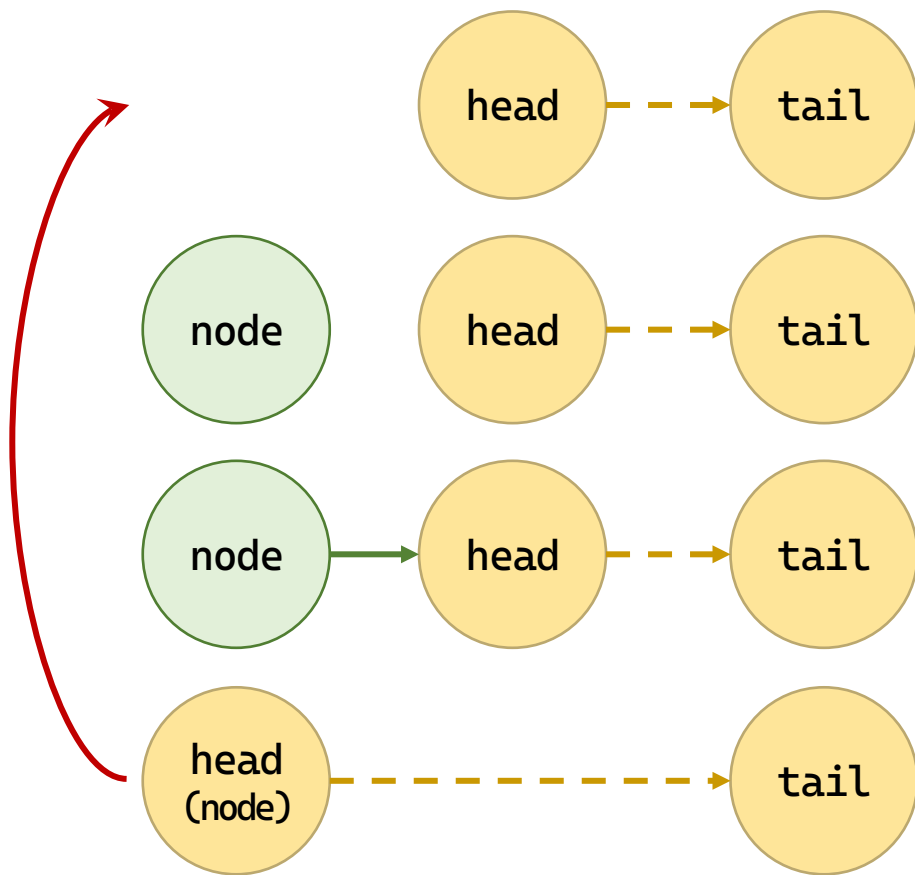
链表：创建列表（头插法）

• 图示

1. head为NULL无例外

2. node 生成失败时，报错、销毁

主体步骤



```
node = (Node*)malloc(sizeof(Node));  
node->data = value;
```

```
node->next = head;
```

```
head = node;
```

```

LinkedList create_from_head(void) {
    LinkedList head = NULL;
    Node* node;
    int value;
    printf("请输入元素，输入非数字结束：\n");
    while (scanf("%d", &value) == 1) {
        node = (Node*)malloc(sizeof(Node));
        if (node == NULL) {
            printf("内存分配失败。\\n");
            free_list(&head);
            clear_input_line();
            return NULL;
        }
        node->data = value;
        node->next = head;
        head = node;
    }
    clear_input_line();
    return head;
}

```

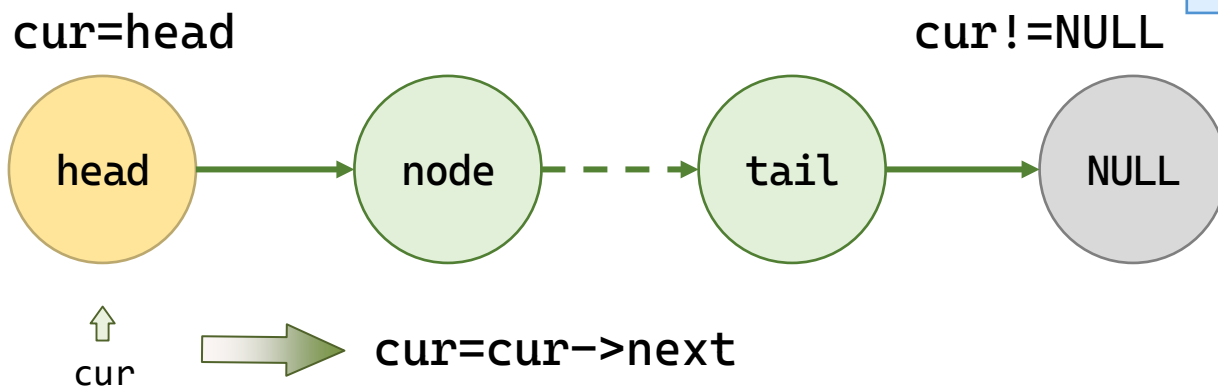
链表：遍历链表

• 按顺序遍历数组和链表比较

功能	数组	链表
定位到第一个元素	$i=0$	$cur=head$
判断最后一个元素	$i < N$	$cur \neq NULL$
移动到下一个元素	$i++$	$cur=cur \rightarrow next$

```
i = 0;
while (i < N) {
    ...;
    i++;
}
```

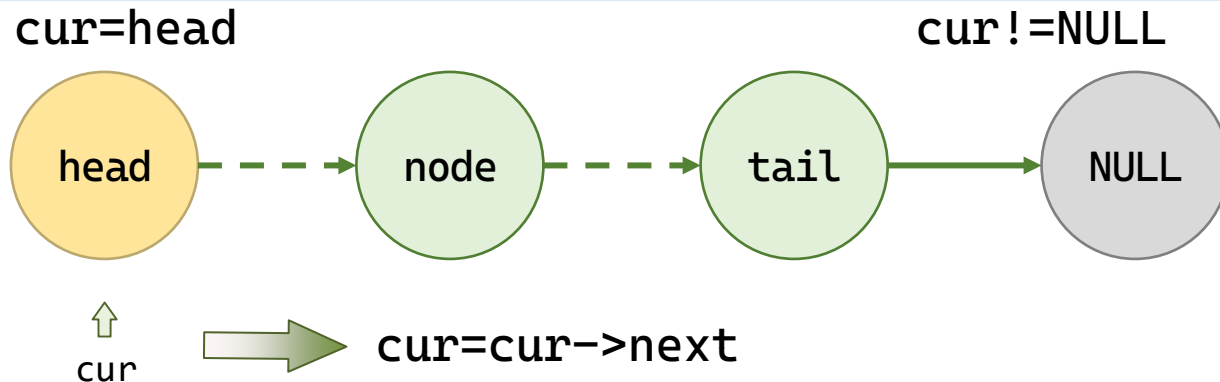
```
cur = head;
while (cur != NULL) {
    ...;
    cur = cur->next;
}
```



链表：计算元素个数

• 示例代码

```
int list_length(LinkList head) {  
    int length = 0;  
    Node* current = head;  
    while (current != NULL) {  
        length++;  
        current = current->next;  
    }  
    return length;  
}
```



链表：定位元素

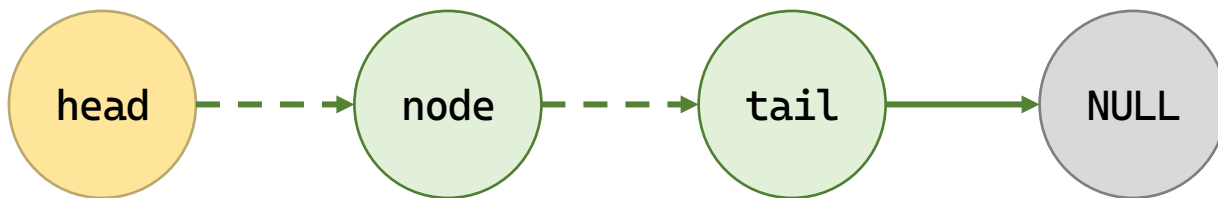
• 示例代码

```
Node* get_node(LinkList head, int index) {  
    int i;  
    Node* current = head;  
    for (i = 0; current != NULL && i < index; i++)  
        current = current->next;  
    return current;  
}
```

点数，数到了就停止

cur=head

cur!=NULL



↑
cur



cur=cur->next

链表：查找元素

• 示例代码

```
Node* locate_node(LinkList head, int value, int* index) {  
    int i;  
    Node* current = head;  
    i = 0;  
    while (current != NULL && current->data != value) {  
        current = current->next;  
        i++;  
    }  
    if (index != NULL) {  
        *index = i;  
    }  
    return current;  
}
```

先有cur不为空才有cur->data

停止条件不是点数，而是找到了

如果未找到不应有副作用，
此时cur是NULL，不需要特殊处理

链表：显示链表

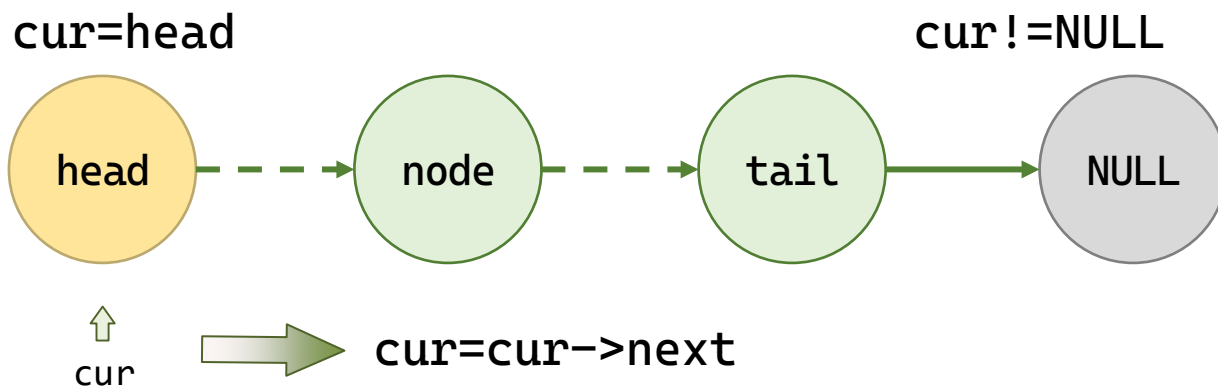
- 示例代码

- 遍历列表

- 输出内容

```
Node* current = head;  
while (current != NULL) {  
    current = current->next;  
}
```

```
printf("%4zu\t%8d\t%016p\t%016p\n", ++i, current->data,  
      (void*)current->next, (void*)current);
```



```

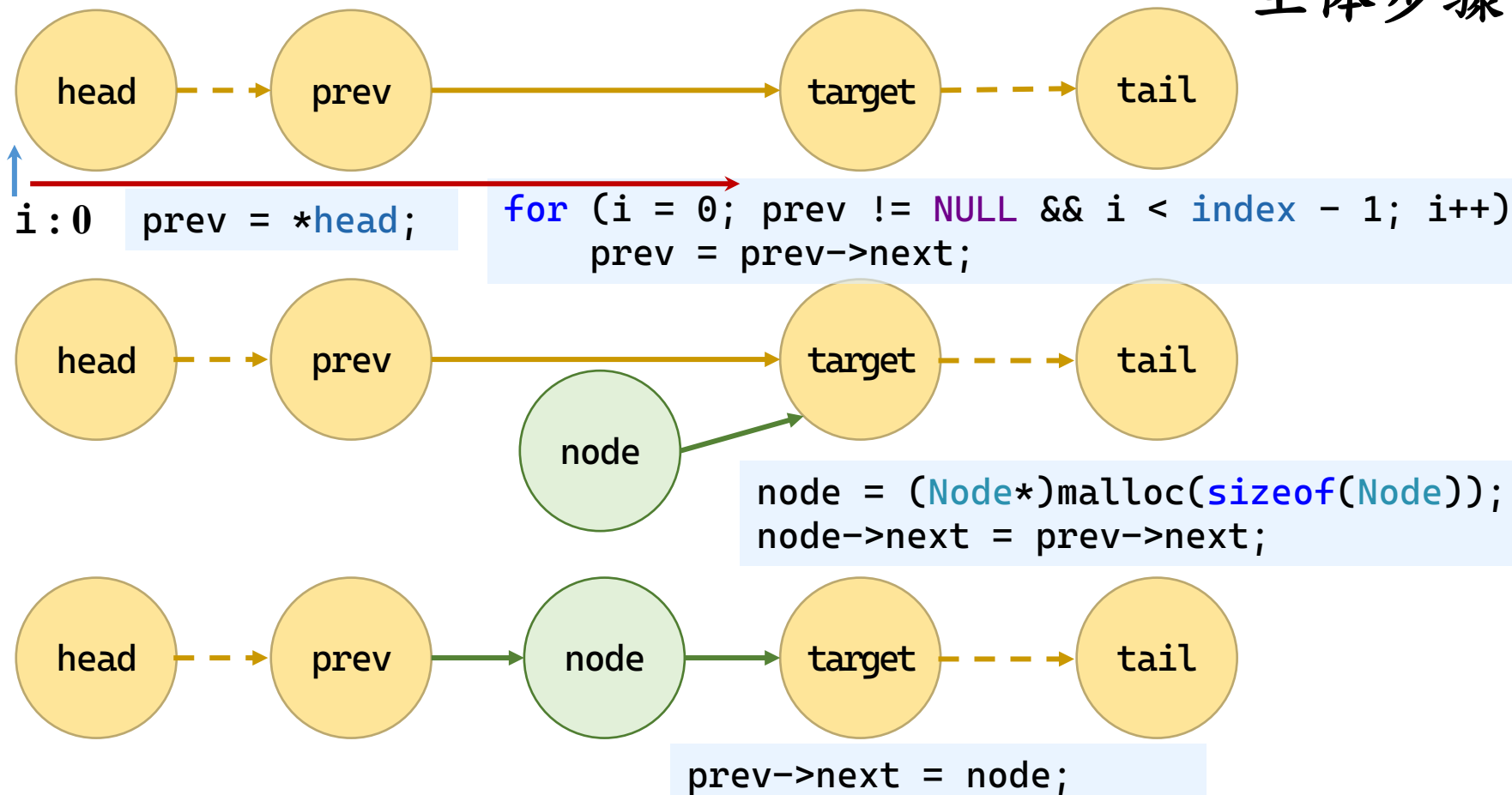
int display_list(LinkList head) {
    Node* current = head;
    size_t i = 0;
    if (current == NULL) {
        fputs("这是个空表。\\n", stderr);
        return OK;
    }
    printf("%-6s\\t%-8s\\t%-16s\\t%-16s\\n", "ID", "数据", "后续", "
    地址");
    printf("====\\t====\\t=====\\t=====
    ====\\n");
    while (current != NULL) {
        printf("%4zu\\t%8d\\t%016p\\t%016p\\n", ++i, current->data,
            (void*)current->next, (void*)current);
        current = current->next;
    }
    return OK;
}

```

链表：插入元素

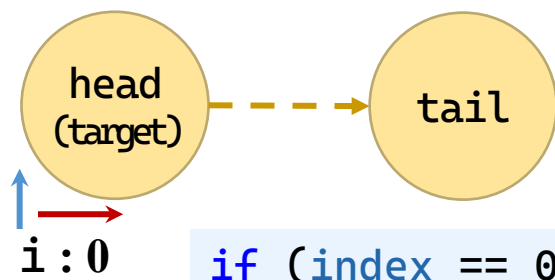
• 插入元素：有前一节点

主体步骤

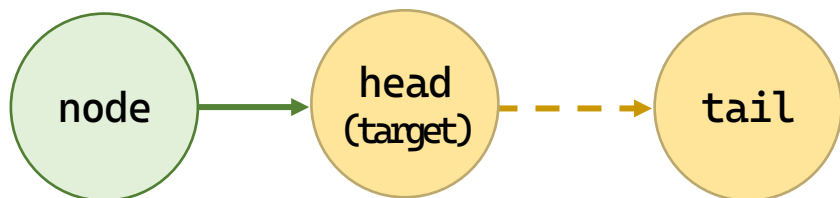


链表：插入元素

• 插入元素：在头节点插入



```
if (index == 0)
```



```
node->next = *head;
```



```
*head = node;
```

特殊情况

1. *head为NULL无例外
2. head为NULL判定
3. 错误参数index判定
4. node 生成失败判定

```

int insert_list(LinkList* head, int index, int value) {
    int i;
    Node* prev;
    Node* node;
    if (head == NULL)
        return NULL_LIST;
    if (index < 0)
        return NOT_FOUND;
    if (index == 0) {
        node = (Node*)malloc(sizeof(Node));
        if (node == NULL)
            return ERROR;
        node->data = value;
        node->next = *head;
        *head = node;
        return OK;
    }
}

```

```

prev = *head;
for (i = 0; prev != NULL && i < index - 1; i++) {
    prev = prev->next;
}
if (prev == NULL) {
    return NOT_FOUND;
}

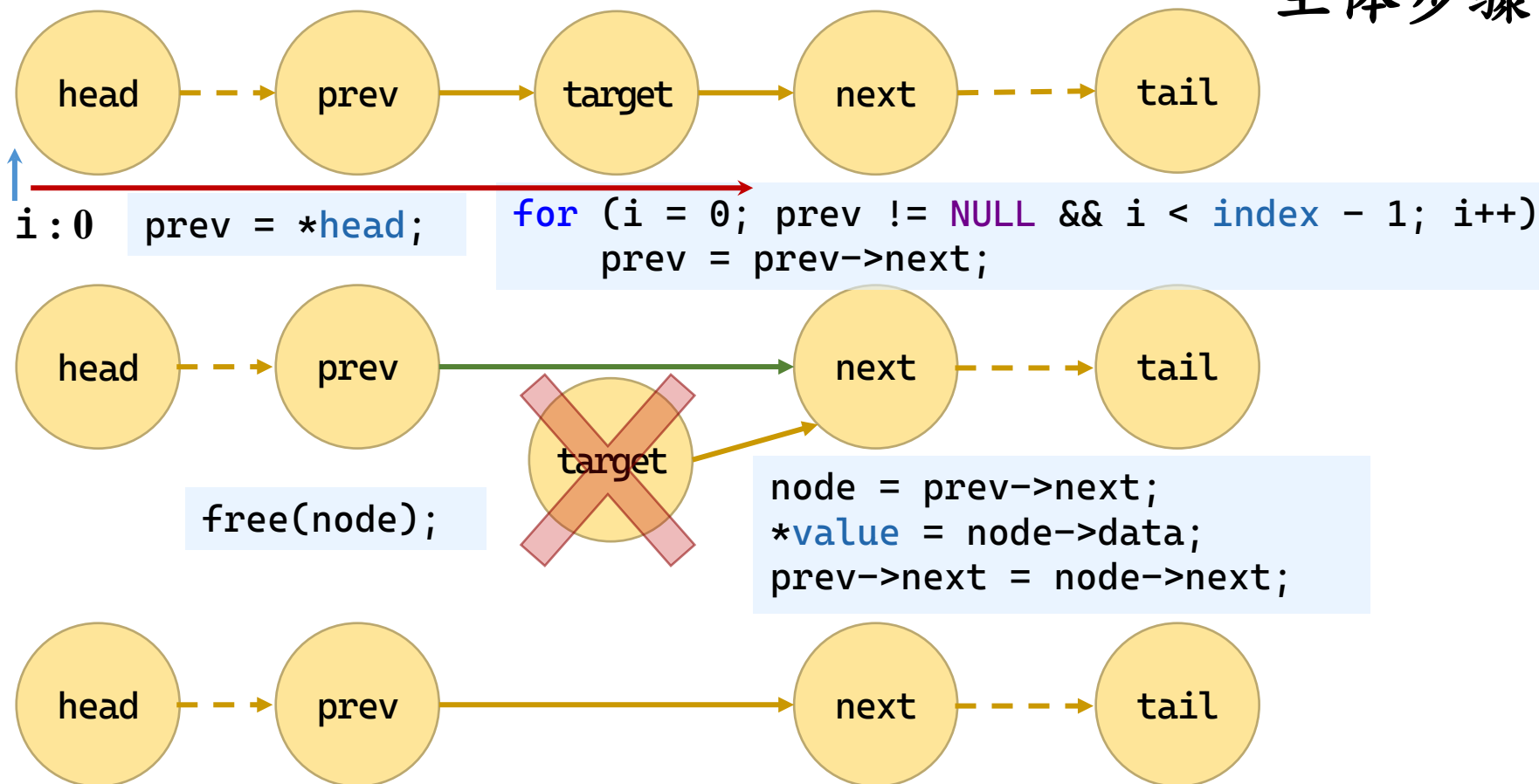
node = (Node*)malloc(sizeof(Node));
if (node == NULL) {
    return ERROR;
}
node->data = value;
node->next = prev->next;
prev->next = node;
return OK;
}

```

链表：删除元素

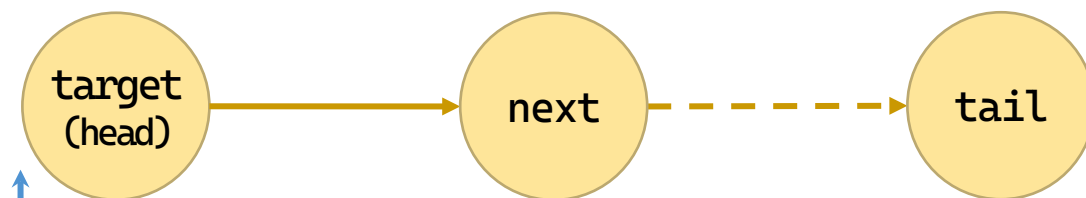
• 删除元素：有前一节点

主体步骤



链表：删除元素

• 删除元素：在头节点删除



$i: 0$ `if (index == 0)`



```
node = *head;  
*value = node->data;  
*head = node->next;
```

```
free(node);
```



特殊情况

1. *head为NULL判定
2. head为NULL判定
3. 错误参数index判定

```

int delete_list(LinkList* head, int index, int* value) {
    int i;
    Node* prev;
    Node* node;

    if (head == NULL || value == NULL || *head == NULL
        || index < 0) {
        return ERROR;
    }

    if (index == 0) {
        node = *head;
        *value = node->data;
        *head = node->next;
        free(node);
        return OK;
    }
}

```

在头删除的情况：
调整结点链接，当前链表头后移；
释放原链表头。

```

prev = *head;
for (i = 0; prev != NULL && i < index - 1; i++) {
    prev = prev->next;
}
if (prev == NULL || prev->next == NULL) {
    return NOT_FOUND;
}

```

寻找插入位置的前一结点
如果不存在该位置报错

```

node = prev->next;
*value = node->data;
prev->next = node->next;
free(node);
return OK;

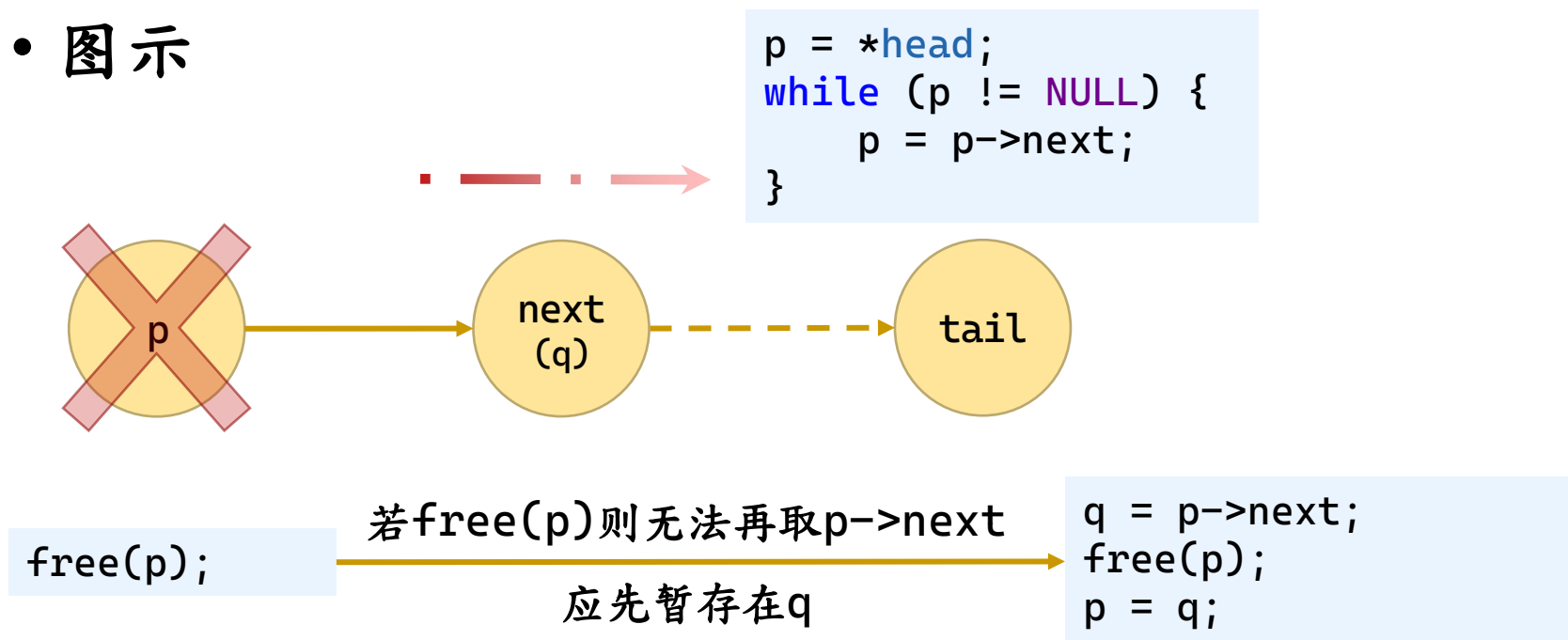
```

一般删除的情况：
调整结点链接，前一结点跳过待删结点，后续为待删结点的后续；
释放待删结点。

链表：销毁

- 由malloc()开辟的空间在程序终止时释放
 - 但程序员不能放任这种情况，应调用free()释放

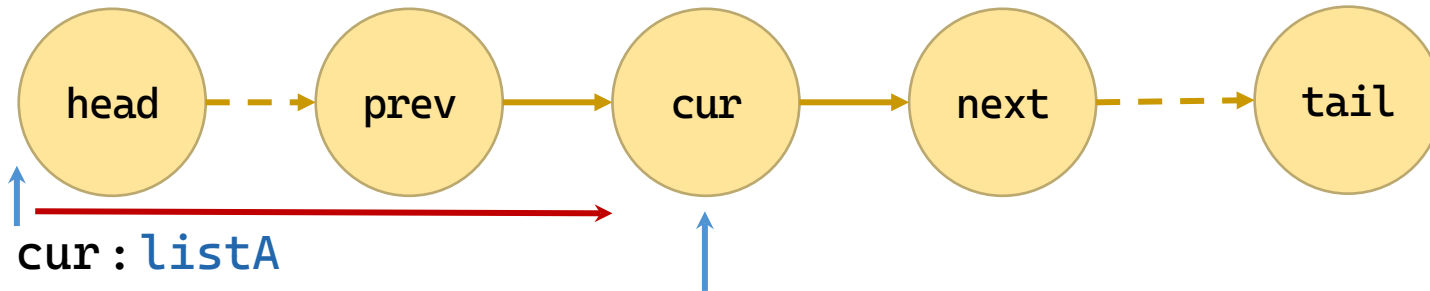
- 图示



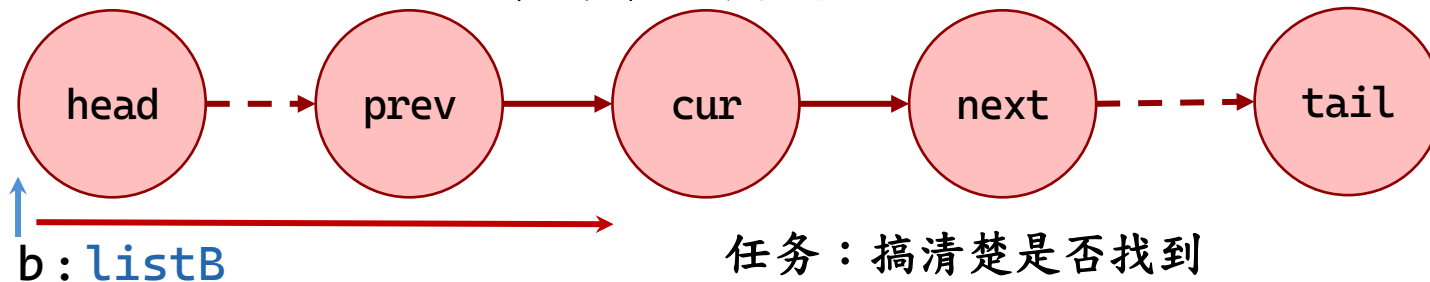
```
void free_list(LinkList* head) {  
    Node* p, * q;  
    if (head == NULL)  
        return;  
    p = *head;  
    while (p != NULL) {  
        q = p->next;  
        free(p);  
        p = q;  
    }  
    *head = NULL;  
}
```

链表：求差集

• 求两个链表A、B的差集A-B



是否找到？找到，则删除；
未找到，则不处理。



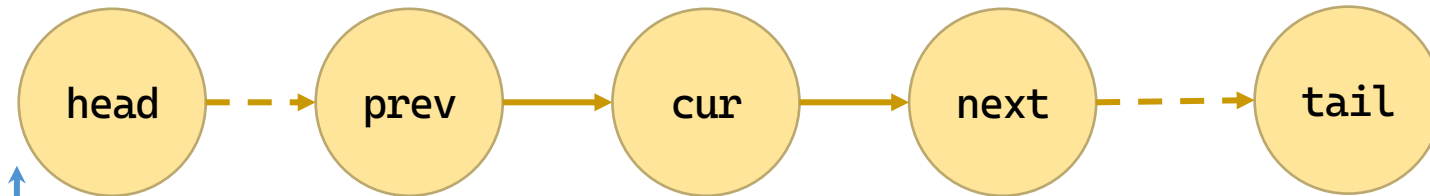
任务：搞清楚是否找到

```
while (b != NULL) {  
    // ...  
    b = b->next;  
}
```

```
if (current->data == b->data) {  
    found = 1;  
    break;  
}
```

链表：求差集

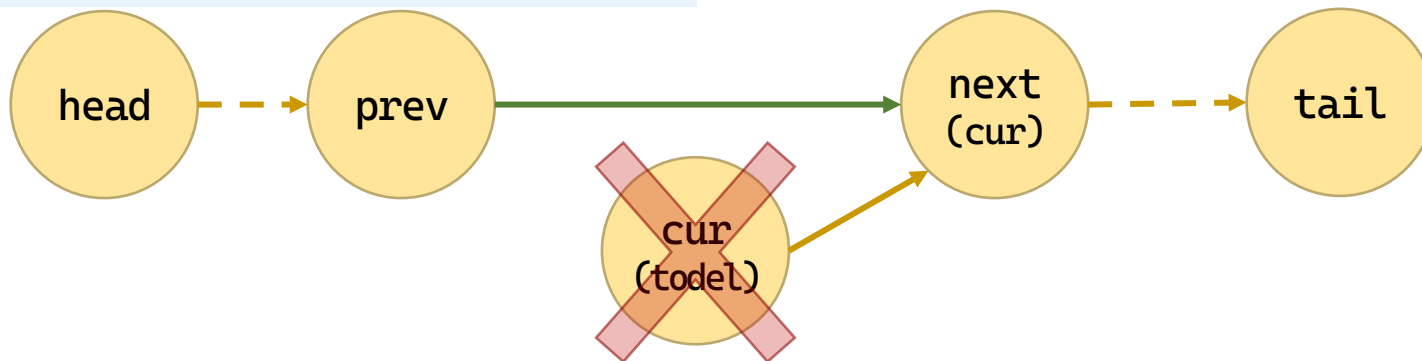
- 如果找到了，则删除，前导不动；否则，都动



cur: listA

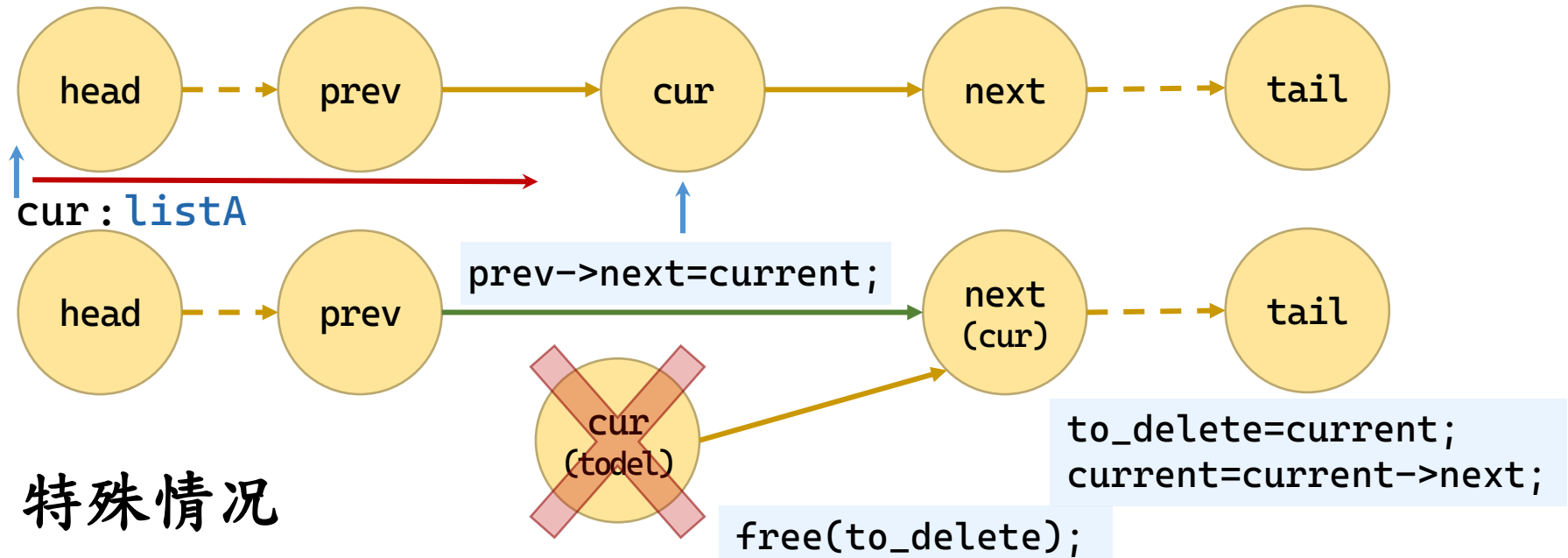
```
while (current != NULL) {  
    // found cur in List B?  
    prev = current;  
    current = current->next;  
}
```

```
if (found) {  
    to_delete = current;  
    current = current->next;  
    prev->next = current;  
    free(to_delete);  
}
```



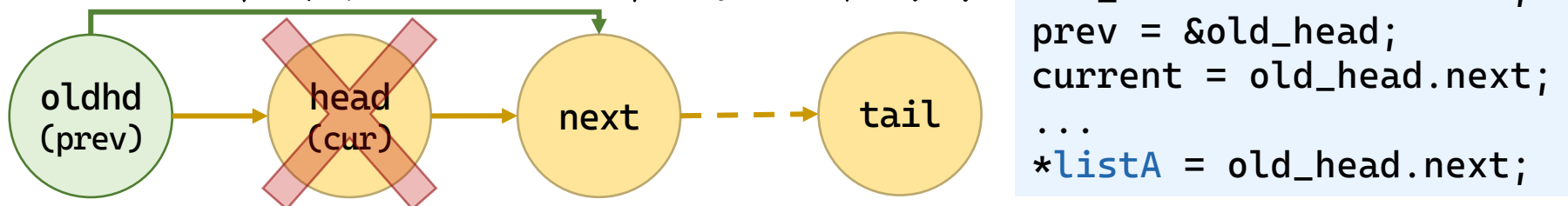
链表：求差集

- 如果找到了，则删除，前导不动



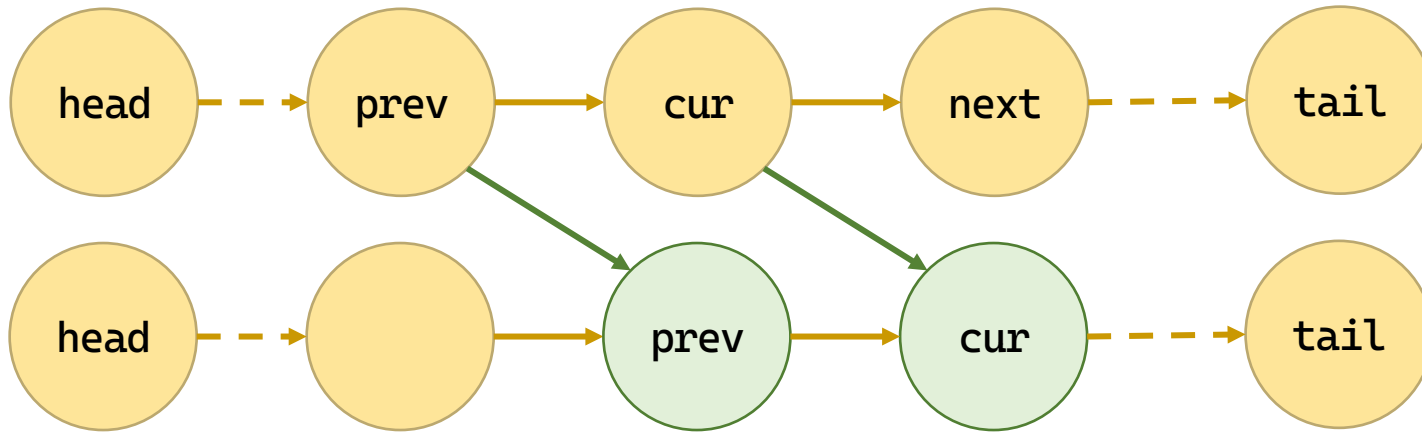
- 特殊情况

– 头指针删除：为头指针设置一个前导



链表：求差集

- 如果找不到，则不删除，前导前进



```
while (current != NULL) {  
    // found cur in List B?  
    prev = current;  
    current = current->next;  
}
```

```
if (found) {  
    ...  
} else {  
    prev = current;  
    current = current->next;  
}
```

```

void difference_list(LinkList* listA, LinkList* listB) {
    Node old_head;
    Node* prev, * current, * b, * to_delete;
    int found;
    if (listA == NULL || *listA == NULL)
        return;
    old_head.next = *listA;
    prev = &old_head;
    current = old_head.next;
    while (current != NULL) {
        found = 0;
        b = (listB != NULL) ? *listB : NULL;
        while (b != NULL) {
            if (current->data == b->data) {
                found = 1;
                break;
            }
            b = b->next;
        }
    }
}

```

```

    }
    if (found) {
        to_delete = current;
        current = current->next;
        prev->next = current;
        free(to_delete);
    } else {
        prev = current;
        current = current->next;
    }
}
*listA = old_head.next;
}

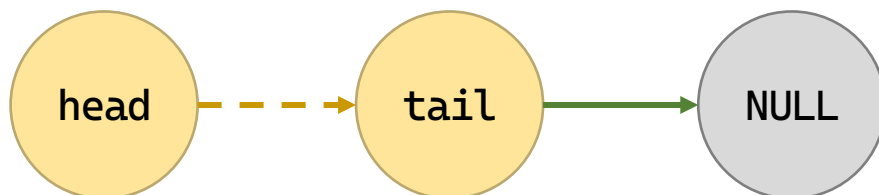
```

目录

	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表
	6	链表的应用

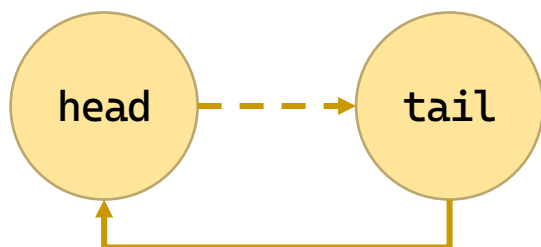
循环链表

- 普通链表尾部的下一节点是空指针



```
tail->next = NULL;
```

- 循环链表尾部的下一节点是链表头

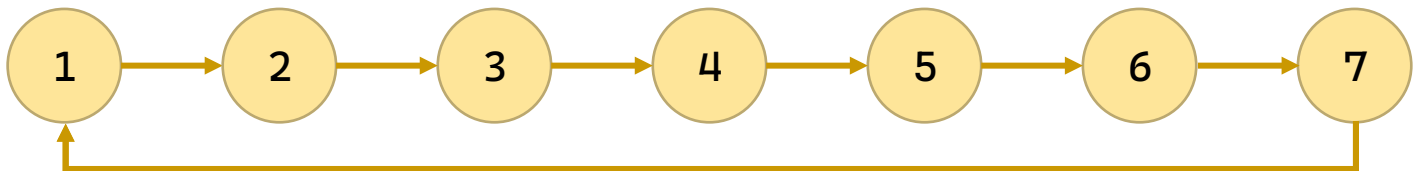


```
tail->next = head;
```

循环链表：应用

• 例2-5 约瑟夫环

- 设有 n 个数构成一个环链，要求从第 k 个数开始数数，数到 m 的那个数被弹出，然后从该数的下一个数重新数数，数到 m 的那个数又被弹出，如此重复，直到所有的数均被弹出为止。输出这些数弹出的序列。
- 例如： $n=7$ ， $k=2$ ， $m=3$
- 输出：4 7 3 1 6 2 5



循环链表：应用

- 主体步骤

- 找到第 k 个节点

```
current = head;  
for (count = 1; count < k; count++) {  
    current = current->next;  
}
```

- 数到第 m 个节点

```
for (count = 1; count < m; count++) {  
    current = current->next;  
}
```

- 打印并删除数到的节点

```
to_delete = current;  
current = current->next;  
prev->next = current;  
printf("%d -> ", to_delete->data);  
free(to_delete);
```

需要用到pre，
在上面步骤要维护pre



```
prev = current;  
current = current->next;
```

- 重复至链表已空

- 应判断只剩一个节点

```
while (current->next != current) {  
    ...  
}
```

循环链表：初步代码

```
void josephus_ring(LinkList head, int k, int m) {
    int count;
    Node *prev, *current, *to_delete;
    current = head;
    for (count = 1; count < k; count++) {
        prev = current;
        current = current->next;
    }
    while (current->next != current) {
        for (count = 1; count < m; count++) {
            prev = current;
            current = current->next;
        }
        to_delete = current;
        current = current->next;
        prev->next = current;
        printf("%d -> ", to_delete->data);
        free(to_delete);
    }
}
```

```

#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* next;
} Node;
typedef Node* LinkList;
LinkList create_list(int n) {
    if (n <= 0)
        return NULL;
    Node* head = (Node*)malloc(sizeof(Node));
    head->data = 1;
    Node* current = head;
    for (int i = 2; i <= n; i++) {
        current->next = (Node*)malloc(sizeof(Node));
        current = current->next;
        current->data = i;
    }
    current->next = head; // 尾指向首, 连成环
    return head;
}

```

```

int main(void) {
    /* 定义测试参数表 */
    int node_counts[] = {0, 1, 5, 5, 0, 5, 5, 5, 5};
    int k_vals[]       = {1, 1, 1, -2, 0, 45, 1, 1, 1};
    int m_vals[]       = {3, 3, 3, 3, 3, 3, -4, 0, 80};
    int total_tests = sizeof(node_counts) /
        sizeof(node_counts[0]);

    /* 使用一个极其简单的 for 循环驱动全部测试例 */
    for (int i = 0; i < total_tests; i++) {
        printf("\n节点数: %d | k = %d | m = %d\n", i + 1,
            node_counts[i], k_vals[i], m_vals[i]);
        josephus_ring(create_list(node_counts[i]), k_vals[i],
            m_vals[i]);
    }

    return 0;
}

```

循环链表：通过测试完善代码

- 先确定定义域，再定义测试域

参数名	取值范围	测试用例
head	任意链表	空表、一个节点表、五节点表
k	任意整数	负数、0、小正数、大正数
m	任意整数	负数、0、小正数、大正数

- 测试发现错误情况

- error C4703: 使用了可能未初始化的本地指针变量“prev”
- 空表时，在while (current->next != current)出空指针

- 应对：

```
if (head == NULL) return;
```

- $k \leq 0$ 或 $m \leq 0$ ，出现prev为空错误

```
if (head == NULL || k < 1 || m < 1)
    return;
```

循环链表：通过测试完善代码

- 测试发现错误情况

- 错误：未输出最后一个节点

【测试用例】节点数: 5 | 起始位置 k: 1 | 每次报数 m: 3
3 -> 1 -> 5 -> 2 ->

- 应对

```
printf("%d\n", current->data);  
free(current);
```

- 错误： $k=1$ 或 $m=1$ ，出现prev为空错误

```
int len = 1;  
Node* tail = head;  
while (tail->next != head) {  
    len++;  
    tail = tail->next;  
}  
prev = tail;  
k = (k - 1) % len + 1;  
m = (m - 1) % len + 1;
```

```

void josephus_ring(LinkList head, int k, int m) {
    if (head == NULL) {
        printf("链表为空, 无法执行约瑟夫环模拟。\\n");
        return;
    }
    int len = 1;
    Node* tail = head;
    while (tail->next != head) {
        len++;
        tail = tail->next;
    }
    Node* prev = tail;
    if (k < 1)
        k = 1;
    else
        k = (k - 1) % len + 1;
    if (m < 1)
        m = 1;
}

```

```

Node* current = head;
for (int count = 1; count < k; count++) {
    prev = current;
    current = current->next;
}
printf("弹出序列: ");
while (current->next != current) {
    for (int count = 1; count < m; count++) {
        prev = current;
        current = current->next;
    }
    Node* to_delete = current;
    current = current->next;
    prev->next = current;
    printf("%d -> ", to_delete->data);
    free(to_delete);
}
printf("%d\n", current->data);
free(current);
}

```

时间复杂度分析

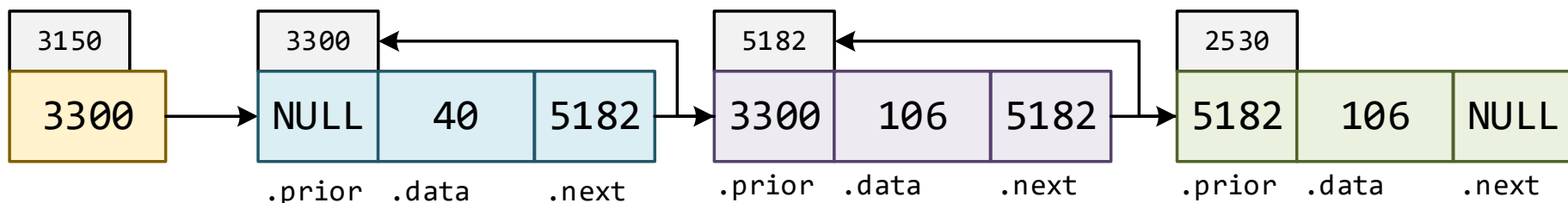
- 定位计数起始位置的时间
 - 通过 $k=(k-1)\%len+1$; 已经从 $O(k)$ 优化为 $O(n)$
- 对所有数据元素，依次完成计数到 m 及弹出数据元素
 - 时间复杂度： $O(m*n)$ 。
- 算法的时间复杂度为：
 - 时间复杂度： $T(m,n)=O(n)+O(m*n)=O(m*n)$

目录

	1	基本概念
	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表

双向链表

- 双向链表增加了前节点域



```
typedef struct DNode {  
    int data;  
    struct DNode* prior;  
    struct DNode* next;  
} DNode, *DLinkedList;
```

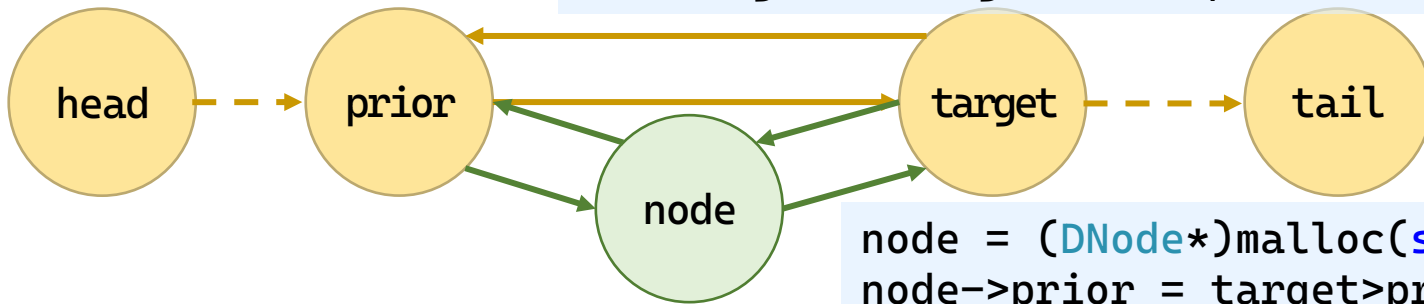
双向链表：插入元素

- 插入元素：有前一节点

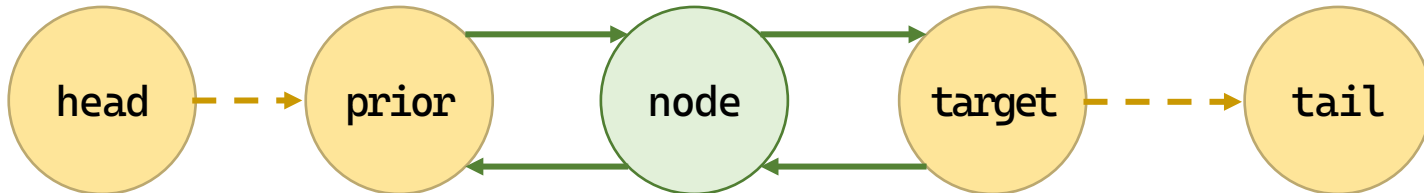
主体步骤



```
i:0 target = *head; for (i = 0; target != NULL && i < index; i++)  
    target = target->next;
```



```
node = (DNode*)malloc(sizeof(DNode));
node->prior = target->prior;
target->prior->next = node;
target->prior = node;
```

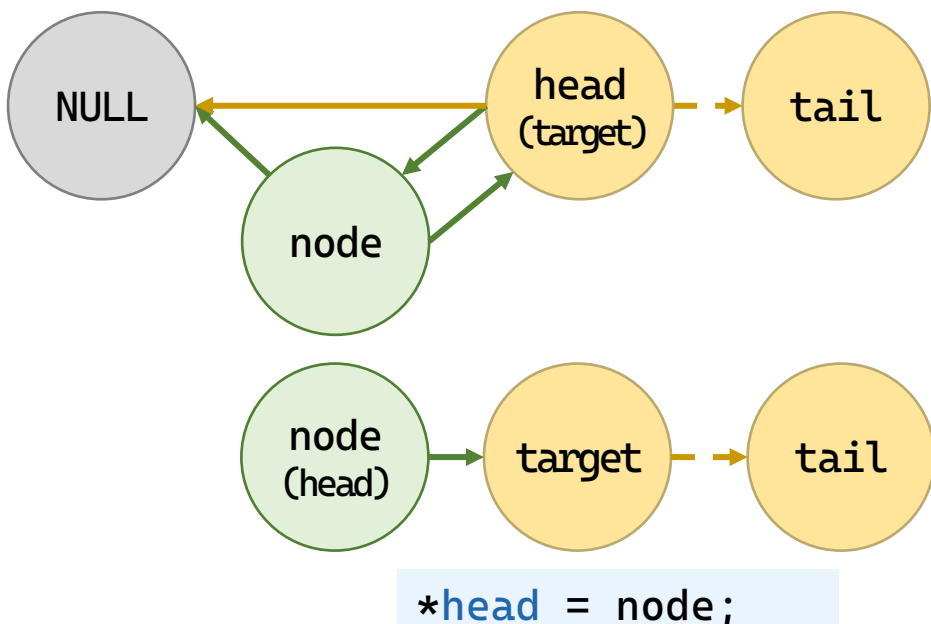
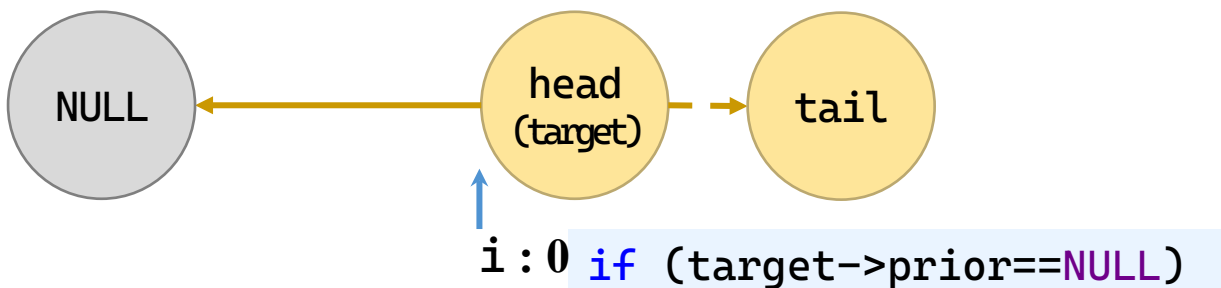


双向链表：插入元素

• 插入元素：在头节点插入

特殊情况

1. *head为NULL例外
2. head为NULL判定
3. 错误参数index判定
4. node 生成失败判定

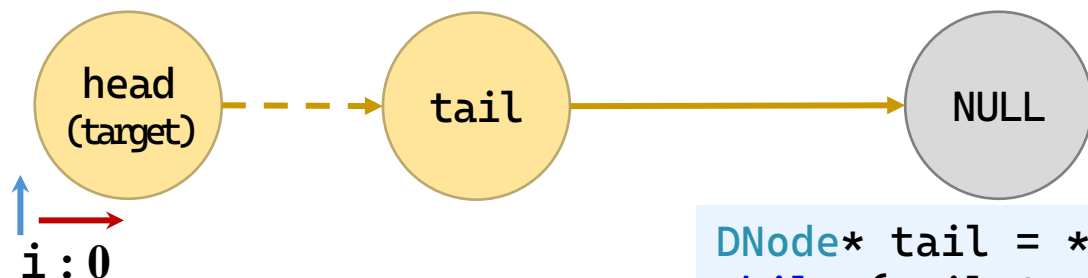


```
node = (DNode*)malloc(sizeof(DNode));  
if (node == NULL)  
    return ERROR;  
node->data = value;  
node->next = target;  
node->prior = target->prior;  
if (target->prior != NULL)  
    target->prior->next = node;  
else  
    *head = node;  
target->prior = node;
```

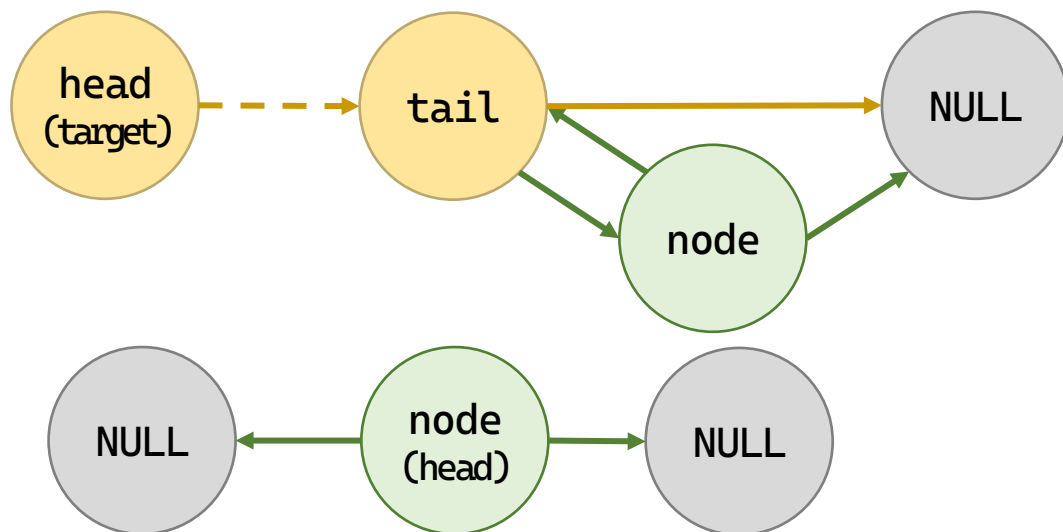
双向链表：插入元素

- 插入元素：找不到的时候尾插

特殊情况



```
DNode* tail = *head;  
while (tail != NULL && tail->next != NULL)  
    tail = tail->next;
```



```
node->next = NULL;  
node->prior = tail;
```

```
if (tail != NULL)  
    tail->next = node;  
else  
    *head = node;
```

```

int insert_list(DLinkedList* head, int index, int value) {
    DNode* target;
    DNode* node;
    int i;
    if (head == NULL)
        return NULL_LIST;
    if (index < 0)
        return NOT_FOUND;
    target = *head;
    for (i = 0; target != NULL && i < index; i++)
        target = target->next;
    if (i < index)
        return NOT_FOUND;
    node = (DNode*)malloc(sizeof(DNode));
    if (node == NULL)
        return ERROR;
    node->data = value;
    if (target != NULL) {
        node->next = target;
    }
}

```

```

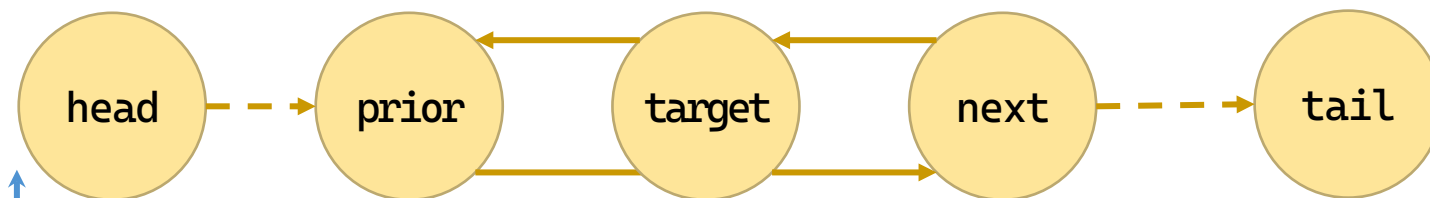
    node->prior = target->prior;
    if (target->prior != NULL)
        target->prior->next = node;
    else
        *head = node;
    }
    target->prior = node;
    return OK;
}
DNode* tail = *head;
while (tail != NULL && tail->next != NULL)
    tail = tail->next;
node->next = NULL;
node->prior = tail;
if (tail != NULL)
    tail->next = node;
else
    *head = node;
return OK;
}

```

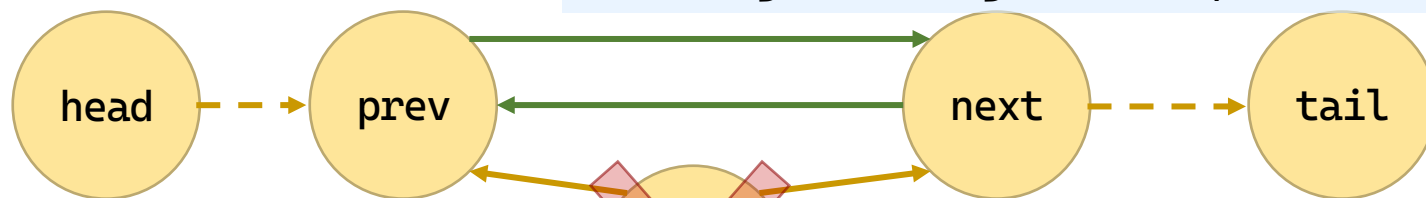
双向链表：删除元素

• 删除元素：有前一节点

主体步骤



$i:0$ `target = *head;` `for (i = 0; target != NULL && i < index; i++)`
`target = target->next;`



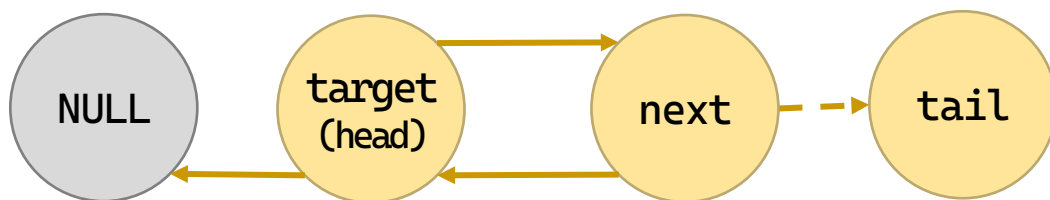
`free(node);`

`target->prior->next = target->next;`
`if (target->next != NULL)`
`target->next->prior = target->prior;`

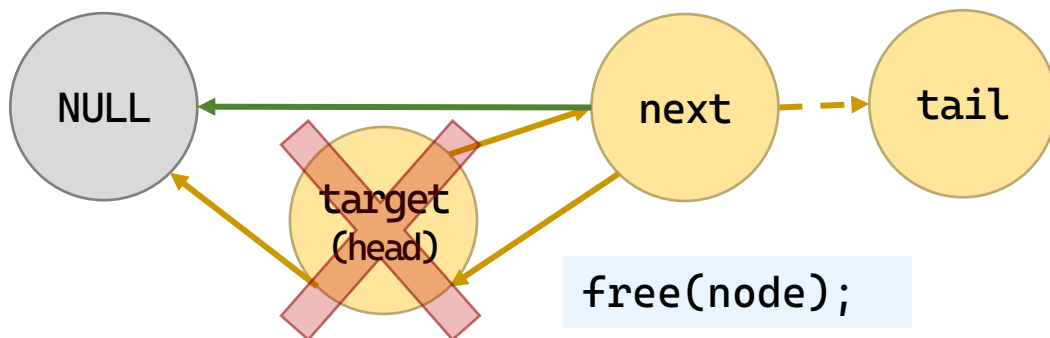


链表：删除元素

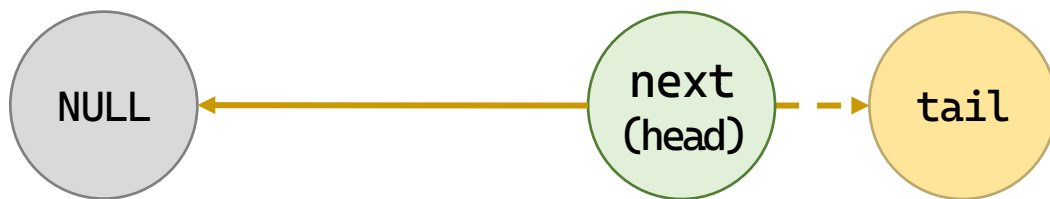
• 删除元素：在头节点删除



```
if (target == *head)
```



```
free(node);
```



```
*head = target->next;
```

特殊情况

1. *head为NULL判定
2. head为NULL判定
3. 错误参数index判定

```
if (*head != NULL)  
    (*head)->prior = NULL;
```

```

int delete_list(DLinkedList* head, int index, int* value) {
    DNode* target;
    int i;
    if (head == NULL || value == NULL || *head == NULL
        || index < 0)
        return ERROR;
    target = *head;
    for (i = 0; target != NULL && i < index; i++)
        target = target->next;
    if (target == NULL)
        return NOT_FOUND;
    *value = target->data;
    if (target == *head) {
        *head = target->next;
        if (*head != NULL)
            (*head)->prior = NULL;
    }
    else {

```

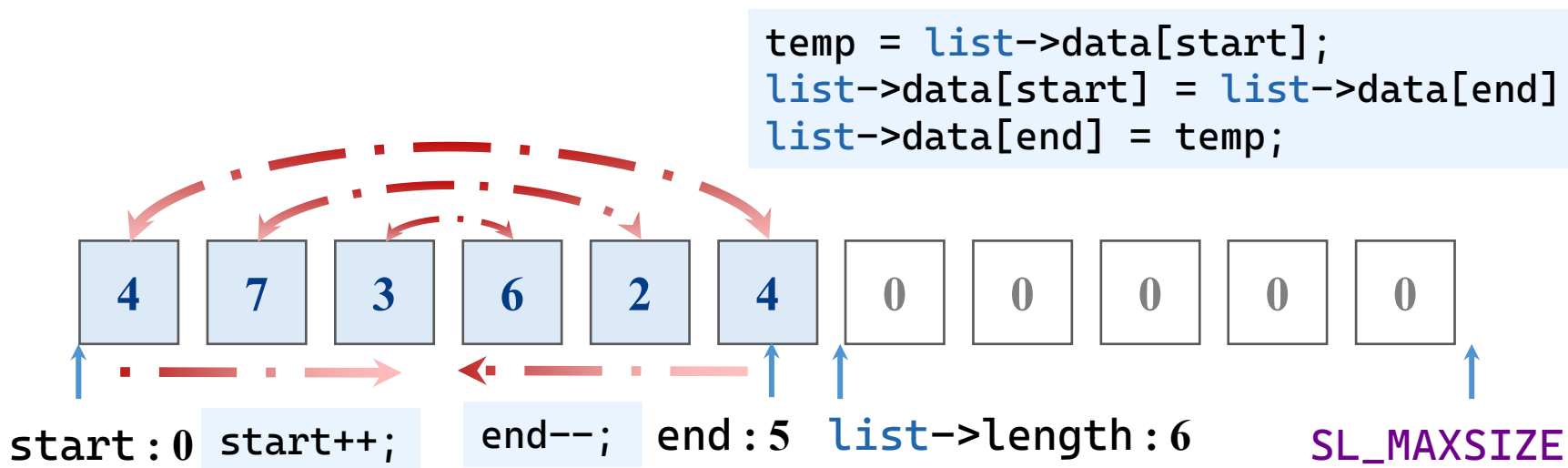
```
    target->prior->next = target->next;
    if (target->next != NULL)
        target->next->prior = target->prior;
}
free(target);
return OK;
}
```

目录

	2	顺序表
	3	单链表
	4	循环链表
	5	双向链表
	6	链表的应用

线性表逆置

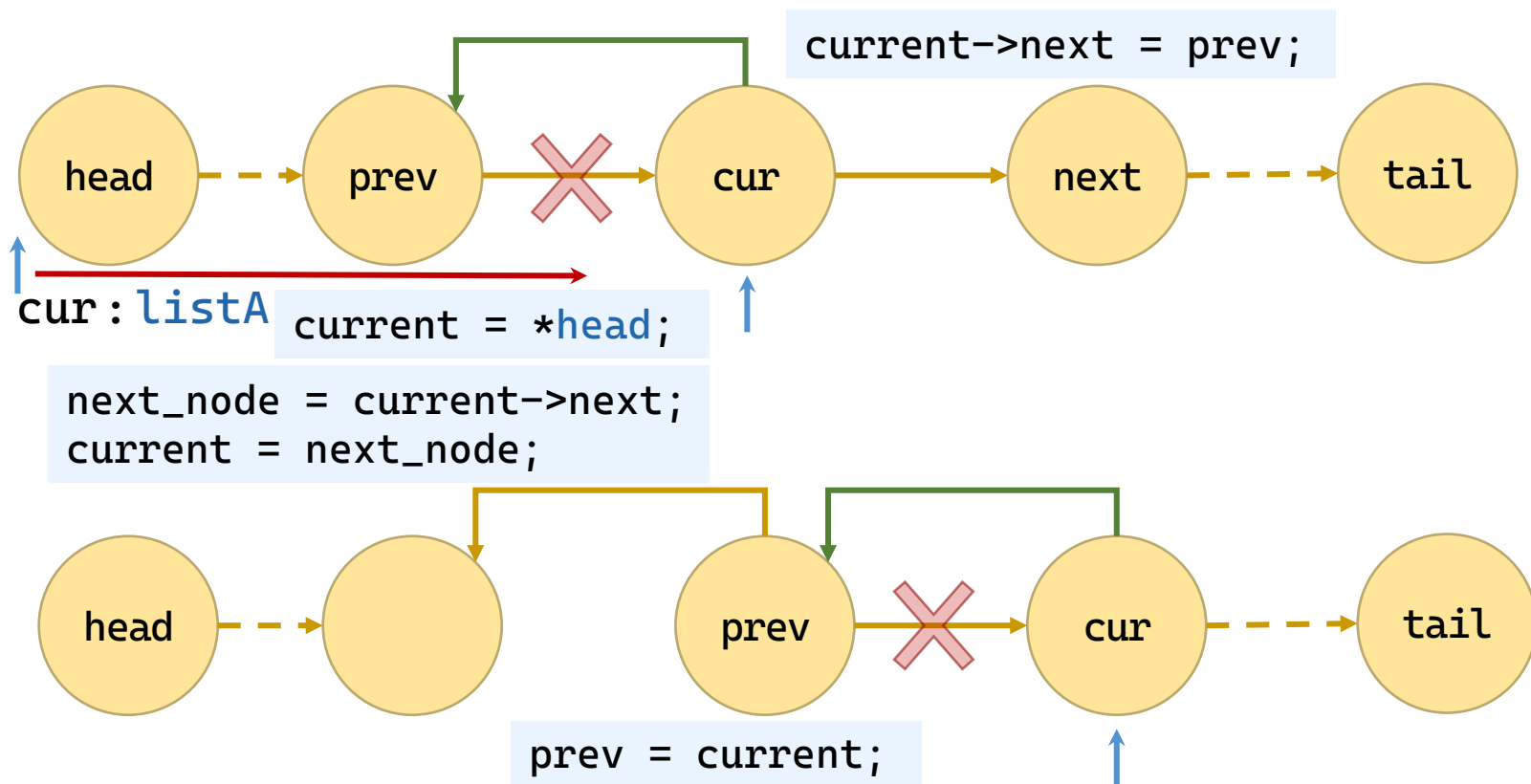
- 例题：设计算法，将线性表L的数据元素逆置。
 - (1) 顺序表：利用随机存取特点，将头尾对称的位置交换。



线性表逆置

• 例题：设计算法，将线性表L的数据元素逆置。

– (2) 链表：在访问过程中，将指针“逆置”。



```

void reverse_list(LinkList* head) {
    Node* prev = NULL;
    Node* current;
    Node* next_node = NULL;
    if (head == NULL || *head == NULL)
        return;
    current = *head;
    while (current != NULL) {
        next_node = current->next;
        current->next = prev;
        prev = current;
        current = next_node;
    }
    *head = prev;
}

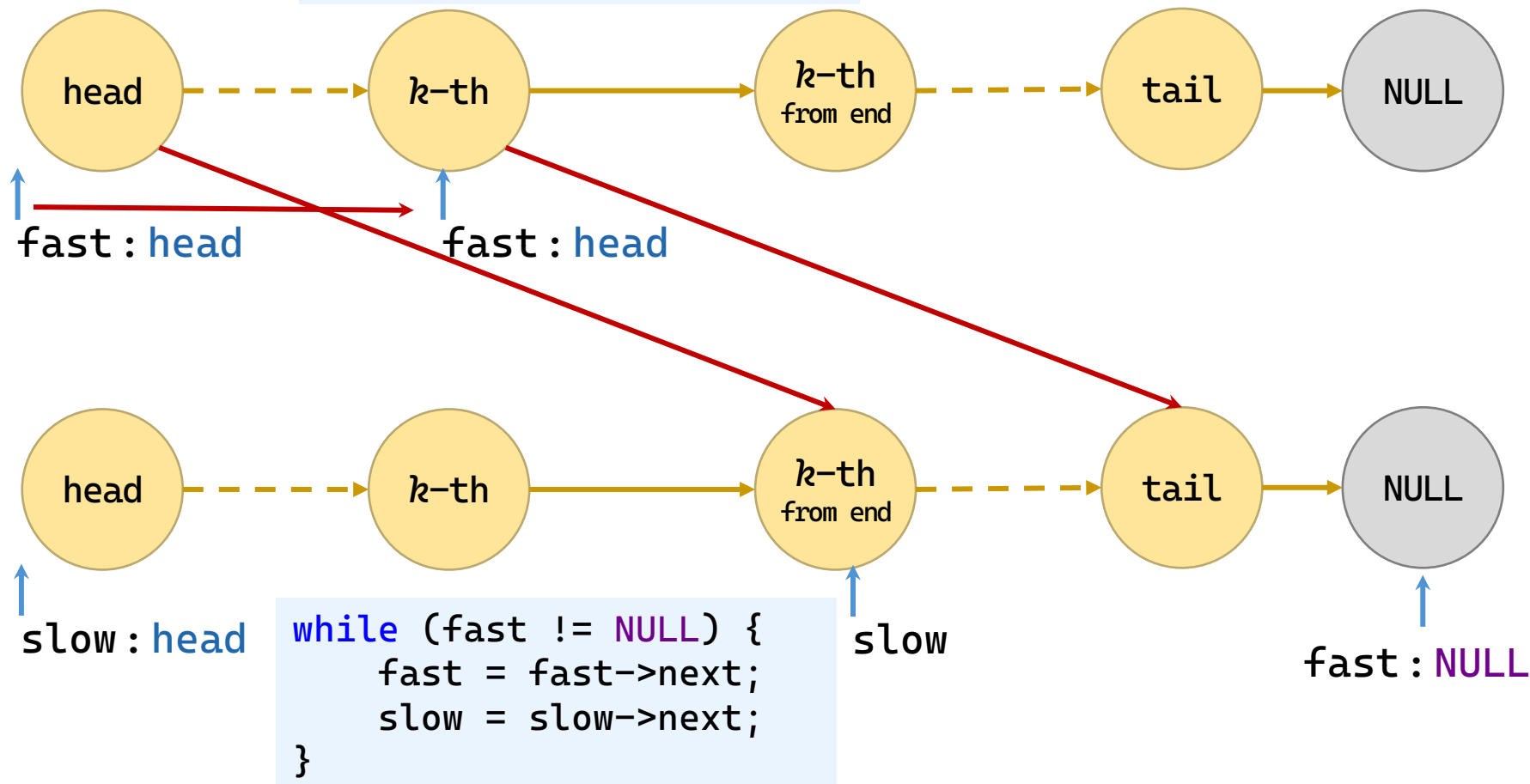
```

查找倒数第 k 个位置上的结点

- 例：给定链表的头指针 L 和一个正整数 k 。试设计一个尽可能高效的算法，用于查找链表 L 中倒数第 k 个位置上的结点。
- 算法思路1：
 - 先计算单链表的长度 n ，再查找第 $n-k+1$ 个结点。
 - 算法的时间复杂度为 $O(2n-k)$ 。
- 算法思路2：（更优）
 - 快指针先走 k 步，然后快指针和慢指针同时走 $n-k$ 步。
 - 算法的时间复杂度为 $O(n)$ 。

查找倒数第 k 个位置上的结点

```
for (i = 0; i < k; i++)  
    fast = fast->next;
```



```

Node* find_kth_from_end(LinkList head, int k) {
    Node* fast = head;
    Node* slow = head;
    int i;
    if (head == NULL || k <= 0)
        return NULL;
    for (i = 0; i < k; i++) {
        if (fast == NULL)
            return NULL;
        fast = fast->next;
    }
    while (fast != NULL) {
        fast = fast->next;
        slow = slow->next;
    }
    return slow;
}

```

谢谢观看



廈門大學
XIAMEN UNIVERSITY



信息学院 黄 焯
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang